# CareShare

**Medical Equipment Sharing Bank**

**A PROJECT REPORT**

***Submitted by***

**Bilakhiya Farhan**

**240905091002**

***In partial fulfillment for the award of the degree of***

**BACHELOR OF TECHNOLOGY**

***In***

**INFORMATION TECHNOLOGY**

**GYANMANJARI INNOVATIVE UNIVERSITY**
**BHAVNAGAR**

**Academic Year -2026**

# GYANMANJARI INNOVATIVE UNIVERSITY

*Survey No.30, Sidsar Road, near Iscon Eleven*

*Bhavnagar-364001, Gujarat*

## CERTIFICATE

This is to certify that the project report entitled **"CareShare – Medical Equipment Sharing Bank"** is a bonafide record of the work carried out by **Bilakhiya Farhan** (Enrollment No. 240905091002) of the Department of Information Technology, Gyanmanjari Innovative University (GMIU), in partial fulfilment of the requirements for the degree of **Bachelor of Technology in Information Technology** during the academic year –2026.

The work embodied in this report has been carried out under my supervision and, to the best of my knowledge, has not been submitted to any other university or institute for the award of any degree or diploma.

| | |
|---|---|
| **Prof. Japan M. Mavani** | **Prof. Dhaval R. Chandarana** |
| **Internal Guide** | **Head of the Department** |

**GYANMANJARI INNOVATIVE UNIVERSITY**

***Survey No.30, Sidsar Road, near Iscon Eleven***

***Bhavnagar-364001, Gujarat***

# DECLARATION

We hereby declare that the project report entitled **"CareShare – Medical Equipment Sharing Bank"**, submitted to the Department of Information Technology, Gyanmanjari Innovative University (GMIU), in partial fulfilment of the requirements for the degree of Bachelor of Technology in Information Technology, is a record of original work done by us under the guidance of **Prof. Japan M. Mavani**.

We further declare that the work reported in this document has not been submitted, in part or in full, for the award of any other degree or diploma of this or any other university or institute. Wherever the work of others has been used, it has been duly acknowledged in the references.

Bilakhiya Farhan
Enrollment No.: 240905091002

Signature: _____

# Acknowledgement

We would like to express our sincere gratitude to **Prof. Japan M. Mavani**, our Internal Guide, for his valuable guidance, support, and encouragement throughout the development of this project. His expertise and insights have played a crucial role in shaping the direction of our work, helping us overcome challenges, and refining our ideas into a well-structured project.

We are also deeply thankful to **Prof. Dhaval R. Chandarana**, **Head of the Department**, for providing us with the necessary resources, motivation, and an environment that fosters learning and innovation. His leadership and support have been instrumental in ensuring the successful completion of this project.

Furthermore, we extend our appreciation to our faculty members, colleagues, and friends who have contributed in various ways, offering valuable feedback and assistance whenever needed. Their encouragement and constructive suggestions have greatly enriched our work.

Lastly, we express our gratitude to our families for their continuous support, patience, and motivation throughout this journey. Their belief in us has been a constant source of inspiration.

This project, **CareShare** , is the result of collective efforts, and we are grateful to everyone who has contributed to its success.

# Abstract

Medical equipment such as wheelchairs, walkers, crutches and patient beds is usually needed only for a short recovery period, yet it is expensive to buy and often lies unused afterwards, while other families struggle to find it quickly. CareShare is a web-based Medical Equipment Sharing Bank that connects donors who have surplus equipment with borrowers who need it temporarily.

The system is a three-tier application. The client is a single-page application built with React, TypeScript, Vite and Tailwind CSS. The server is a RESTful API built with Node.js, Express and TypeScript, exposing 35 endpoints. Data is stored in MongoDB through Mongoose in six collections: users, equipment, borrow requests, return requests, notifications and messages. Authentication uses JSON Web Tokens and bcrypt password hashing, and access is controlled by four roles: borrower, donor, dual-role user and administrator.

Every donated item starts in a Pending Verification state and becomes public only after administrator approval. Borrowing is modelled as a nine-state request lifecycle with a stored status history. At return, the borrower reports the item's condition; the donor or an administrator inspects it, after which the item is made available again, flagged for sanitization or withdrawn if damaged. Notifications, messaging, an administrator dashboard, sanitization records, user management and CSV reports complete the platform.

The report presents the requirements, design, implementation, testing approach and deployment strategy of the system. A critical review of the code identified authorization, validation and testing gaps, which are documented together with recommended corrections and future enhancements.

**Keywords:** medical equipment sharing, community healthcare, React, Node.js, Express, MongoDB, JWT, role-based access control.

# Table of Contents

Acknowledgementiii

Abstractiv

**Chapter-1: Introduction**1

1.1 Project Summary1

1.2 Project Details1

1.3 Project Specification3

1.3.1 General Description3

1.3.2 Functional Specifications3

1.3.3 Administrative and Vendor Capabilities3

1.3.4 Limitations and Future Scope4

1.4 Objectives of the System4

1.4.1 Enable Equipment Donation4

1.4.2 Ensure Quality Through Verification4

1.4.3 Simplify Discovery of Equipment4

1.4.4 Manage the Complete Borrowing Lifecycle4

1.4.5 Protect Hygiene and Equipment Condition4

1.4.6 Provide Communication and Awareness4

1.4.7 Provide Secure, Role-Based Administration5

1.5 Scope of the Project5

1.5.1 Target Users5

1.5.2 Core Functionalities5

1.5.3 Technical Scope5

1.5.4 Limitations5

1.5.5 Future Scope6

**Chapter-2: Literature Review**7

2.1 Evolution7

2.2 Existing Platforms7

2.3 Summary of Literature Review8

**Chapter-3: System Requirement Study**9

3.1 Study of Current System9

3.2 Problems and Weaknesses of Current System9

3.3 User Characteristics9

3.4 Hardware and Software Requirements10

Hardware Requirements10

Software Requirements10
3.5 Constraints10
3.5.1 User Interface10
3.5.2 Communications Interface10
3.5.3 Hardware Interface11
3.5.4 Criticality of the Application11
3.5.5 Safety and Security Considerations11
3.6 Assumptions and Dependencies11
3.6.1 Assumptions11
3.6.2 Dependencies11
**Chapter-4: System Design**13
4.1 Overview of System Workflow13
4.2 Database Design and ER Diagrams16
4.2.1. Database Structure16
4.2.2. Entity-Relationship (ER) Model19
4.3 User Interface and Experience Design (UI/UX)20
4.3.1. User Interface Design Principles20
4.3.2. User Experience Enhancements21
4.4 Class Diagrams and System Components21
4.4.1. Major System Components21
4.4.2. Class Diagram Structure22
**Chapter-5: Implementation and Development**25
5.1 Technology Stack25
5.2 Frontend Implementation25
5.3 Backend Implementation27
5.4 Database Integration29
5.5 Role-Based Access Control30
**Chapter-6: Testing and Deployment**32
6.1 Testing Strategies (Unit, Integration, User Testing)32
6.1.1. Unit Testing32
6.1.2. Integration Testing33
6.1.3. User Acceptance Testing (UAT)33
6.2 Performance Testing & Load Testing34
6.2.1 Load Testing34
6.2.2 Stress Testing35
6.2.3 Database Performance Testing35

6.3 Deployment Strategy (CI/CD, Server Setup, AWS S3, etc.)35
6.3.1 CI/CD Pipeline Setup35
6.3.2 Server Configuration36
6.3.3 AWS S3 Integration36
6.3.4 Deployment Process36
6.4 Challenges Faced During Deployment37
6.4.1 Server Downtime During Migration37
6.4.2 SSL Configuration for Secure Access37
6.4.3 Load Balancing Configuration38
**Chapter-7: Results and Discussions**39
7.1 Key Achievements39
7.2 User Adoption and Feedback39
7.2.1. Student Experience39
7.2.2. Faculty and Administrative Feedback40
7.2.3. Key Areas for Improvement40
7.3 User Feedback and Insights40
7.4 Performance Analysis40
7.5 Limitations41
**Chapter-8: Conclusion and Future Scope**43
8.1 Conclusion43
8.2 Future Scope43
Final Thoughts44
**References**45
**Appendix**46
Appendix A: Running the System and Demonstration Accounts46
Appendix B: Project Directory Structure46
Appendix C: Sample API Exchange47
Appendix D: Screenshots47

# List of Figures

Figure 4.1: Three-tier architecture of CareShare13
Figure 4.2: End-to-end equipment sharing workflow14
Figure 4.3: State diagram of a borrow request15
Figure 4.4: Use-case diagram16
Figure 4.5: Entity-relationship diagram of CareShare20
Figure 4.6: Class diagram of the data models23
Figure 4.7: Class diagram of controllers and middleware24
Figure D.1: Home page47
Figure D.2: Browse Equipment page with filters48
Figure D.3: Equipment Details page and borrow request modal48
Figure D.4: User Dashboard48
Figure D.5: Admin Dashboard48
Figure D.6: Admin Verification queue49

# List of Tables

Table 1.1: Summary of functional specifications3
Table 2.1: Qualitative comparison of existing approaches with CareShare7
Table 3.1: User classes and their characteristics9
Table 3.2: Hardware requirements10
Table 3.3: Software requirements10
Table 4.1: Collection: users17
Table 4.2: Collection: equipment17
Table 4.3: Collection: borrowrequests18
Table 4.4: Collection: returnrequests18
Table 4.5: Collection: notifications18
Table 4.6: Collection: messages19
Table 4.7: Application routes and access level21
Table 4.8: Major system components21
Table 5.1: Technology stack and versions (from package.json)25
Table 5.2: REST API endpoints27
Table 5.3: Permission matrix by role31
Table 6.1: Unit-test candidates and expected behaviour32
Table 6.2: Designed integration test cases33
Table 6.3: User acceptance scenarios34
Table 6.4: Server environment variables36
Table 7.1: Objectives and corresponding implementation39
Table 7.2: Proposed feedback questionnaire40
Table 7.3: Client production build size (from client/dist)41
Table 7.4: Limitations and review findings41
Table A.1: Demonstration accounts created by the seeding utility46

# List of Abbreviations

| Abbreviation | Full form |
|---|---|
| API | Application Programming Interface |
| bcrypt | Adaptive password-hashing algorithm (Blowfish-based) |
| CORS | Cross-Origin Resource Sharing |
| CRUD | Create, Read, Update, Delete |
| CSS | Cascading Style Sheets |
| CSV | Comma-Separated Values |
| CI/CD | Continuous Integration / Continuous Delivery |
| DB | Database |
| ER | Entity-Relationship |
| HTTP(S) | Hypertext Transfer Protocol (Secure) |
| JSON | JavaScript Object Notation |
| JWT | JSON Web Token |
| MERN | MongoDB, Express, React, Node.js |
| ODM | Object-Document Mapper |
| OWASP | Open Worldwide Application Security Project |
| REST | Representational State Transfer |
| RBAC | Role-Based Access Control |
| S3 | Amazon Simple Storage Service |
| SPA | Single-Page Application |
| SSL/TLS | Secure Sockets Layer / Transport Layer Security |
| UAT | User Acceptance Testing |
| UI / UX | User Interface / User Experience |
| URL | Uniform Resource Locator |
| Zod | TypeScript-first schema validation library |

# Chapter 1: Introduction

## 1.1 Project Summary

CareShare is a full-stack web application that works as a community **Medical Equipment Sharing Bank**. It connects people who own medical equipment they no longer need (donors) with patients, caregivers and families who need the same equipment for a limited period (borrowers). The platform is built around the tagline “Share Care. Support Recovery.” and covers the complete lifecycle of a shared item: listing, administrative verification, discovery, borrowing, return, damage inspection and sanitization.

Mobility aids and bedside equipment such as wheelchairs, walkers, crutches, patient beds, oxygen cylinder stands, commode chairs, bedside tables and walking frames are typically needed only during a recovery period of a few weeks or months. After recovery they remain unused in homes, while other families struggle to afford or locate the same items at short notice. CareShare addresses this gap by providing a trusted, moderated and trackable channel for reuse rather than one-time ownership.

The system is implemented as a three-tier application. The presentation tier is a single-page application written in React 18 with TypeScript, Vite and Tailwind CSS. The application tier is a RESTful API written in Node.js and Express with TypeScript. The data tier is MongoDB, accessed through the Mongoose object-data-mapping library. Access to protected functionality is controlled with JSON Web Tokens (JWT), bcrypt password hashing and a role model consisting of four roles: borrower, donor, both (dual role) and admin.

## 1.2 Project Details

**Objective:** To design and develop a secure, moderated web platform that enables the donation, verification, discovery, borrowing, return and sanitization tracking of medical equipment, so that usable equipment is reused within the community and the burden on patients’ families is reduced.

**Key Features:**

- **Public website** – home page with a platform-statistics counter, featured equipment and category cards, equipment browsing with search, category, city, condition and availability filters and server-side pagination, equipment detail page with image

gallery, specifications and sanitization tag, and information pages (About, How It Works, FAQ, Contact).

- **Donation workflow** – a donor submits equipment through a structured form; every new listing starts in the "Pending Verification" state and is invisible to the public until approved by an administrator.
- **Borrow-request workflow** – a nine-state request lifecycle (pending, approved, rejected, cancelled, collected, borrowed, return_requested, returned, completed) with a stored status history for each request.
- **Return and inspection** – the borrower reports the post-use condition and damage details; the donor or an administrator records the inspection result, which automatically updates equipment availability and sanitization status.
- **Communication** – in-app notifications for every key event and a one-to-one messaging thread between donor and borrower.
- **Administrator panel** – dashboard analytics, verification queue, sanitization control, user management (suspend/activate), request and return monitoring, and report generation with CSV export.

**Target Audience:** Individuals and families who need short-term medical equipment, donors who wish to pass on unused equipment, community or hospital volunteers who verify and sanitize equipment, and administrators of a charitable equipment bank.

**Technologies Used:** React 18, TypeScript, Vite, Tailwind CSS, React Router v6, Axios, TanStack Query and Lucide icons on the client; Node.js, Express 4, Mongoose 8, JSON Web Tokens, bcryptjs, Helmet, CORS, express-rate-limit, Multer and Zod on the server; MongoDB as the database.

**Expected Outcomes:** A working end-to-end platform that demonstrates a trustworthy equipment-sharing process: moderated listings, traceable borrowing, damage reporting and hygiene tracking, together with role-based dashboards for every type of user.

**Limitations:** The current implementation uses image URLs rather than uploaded files, has no automated test suite, does not include payment, delivery logistics or real-time push messaging, and contains several server-side authorization gaps that are documented in Section 7.5 and addressed in the future scope.

**Future Enhancements:** Cloud object storage for images, real-time notifications through WebSockets, e-mail and SMS alerts, geo-location based search, a mobile application,

multilingual support, automated tests with a CI/CD pipeline and stricter server-side authorization and validation.

**Long-Term Vision:** To grow CareShare into a network of community equipment banks run in collaboration with hospitals, NGOs and local administrations, where every unused piece of medical equipment can be located, verified and lent within a short distance of a patient in need.

## 1.3 Project Specification

The proposed project, CareShare, is a web-based medical equipment sharing bank that digitizes the donation and lending of durable medical equipment. The following specifications describe the key features and technical scope of the system.

### 1.3.1 General Description

CareShare is delivered as a client–server web application. The browser-based client communicates with the server exclusively through a JSON REST API served under the /api prefix. The system recognizes unauthenticated visitors (guests) and four authenticated roles. Guests can browse and view approved equipment. Registered users act as borrowers, donors or both. Administrators moderate the platform. The server exposes 34 business endpoints plus a health-check endpoint, and persists data in six MongoDB collections.

### 1.3.2 Functional Specifications

**Table 1.1: Summary of functional specifications**

| Module | Specification |
|---|---|
| Authentication | Registration with validated input, login with e-mail and password, JWT-based session (7-day expiry), profile retrieval, password change, suspended-account blocking. |
| Equipment catalogue | Create, read, update and delete listings; eight categories; four condition grades; text search; filters for category, city, condition, availability; sorting by recency, popularity and borrow count; pagination. |
| Borrow requests | Create request with purpose, dates, quantity, pickup preference and address; approve, reject and cancel; controlled status transitions; status history with actor and note. |
| Returns | Borrower return form with post-use condition, damage report and notes; donor/admin inspection (Passed, Needs Inspection, Damaged) with automatic equipment status update. |
| Notifications | Notification generation on request submitted, approved, rejected, equipment verified/rejected, return initiated and inspection updated; mark one/all as read; unread counter. |

| Module | Specification |
|---|---|
| Messaging | Conversation list and message thread between two users, optionally linked to an equipment item; read-state tracking. |
| Administration | Dashboard statistics, equipment verification, sanitization records, user suspension, request/return monitoring, reports with CSV export. |

### 1.3.3 Administrative and Vendor Capabilities

In CareShare the “vendors” are the donors who supply equipment to the bank and the administrators who control its quality. Donors can list equipment, edit or delete their own listings, review incoming requests, approve or reject them and inspect returned items. Administrators can additionally approve or reject any listing, record the sanitization date, notes and proof reference, suspend or reactivate any non-admin account, monitor all requests and returns and view platform-wide analytics. An administrator may also act on any request or return, which gives the bank a safety net when a donor is unavailable.

### 1.3.4 Limitations and Future Scope

The present version is a functional prototype intended for academic demonstration. Equipment photographs are referenced by URL, notifications are pull-based, and the server relies on a single node process. Planned improvements include object storage, real-time channels, e-mail/SMS integration, geo-search, automated testing and deployment automation, as described in Chapter 8.

## 1.4 Objectives of the System

The primary objective of CareShare is to provide a transparent and trustworthy platform through which unused medical equipment can be shared with those who need it. The system aims to fulfill the following objectives:

### 1.4.1 Enable Equipment Donation

Provide a simple form-based process by which any registered donor can list medical equipment together with its condition, specifications, location and sanitization details.

### 1.4.2 Ensure Quality Through Verification

Guarantee that no listing becomes publicly visible before an administrator has reviewed it, by enforcing the Pending Verification state at creation and the approval step in the admin panel.

### 1.4.3 Simplify Discovery of Equipment

Allow borrowers to find suitable equipment quickly through search, filters, sorting and pagination, and to evaluate it through photographs, specifications, location and sanitization status.

### 1.4.4 Manage the Complete Borrowing Lifecycle

Model borrowing as an explicit state machine with a stored history, so that every request can be traced from submission to completion and every actor and note is recorded.

### 1.4.5 Protect Hygiene and Equipment Condition

Record post-use condition and damage reports at return, support donor/admin inspection, and maintain a sanitization status (Pending, Sanitized, Needs Cleaning) for each item.

### 1.4.6 Provide Communication and Awareness

Keep all parties informed through in-app notifications and allow direct donor–borrower communication to coordinate pickup, delivery and return.

### 1.4.7 Provide Secure, Role-Based Administration

Protect user data and platform operations through hashed passwords, token-based authentication, role-based access control and suspension of misbehaving accounts, and give administrators the analytics needed to run the bank.

## 1.5 Scope of the Project

The CareShare project is developed to support the community sharing of durable medical equipment, from listing to return, within a moderated web environment. The scope of this project includes the following aspects:

### 1.5.1 Target Users

- **Guests** – browse the catalogue and read information pages.
- **Borrowers** – patients, caregivers and families who request equipment.
- **Donors** – individuals or organizations that list and lend equipment.
- **Dual-role users** – users who both donate and borrow.
- **Administrators** – staff who verify equipment, record sanitization, manage users and monitor the platform.

### 1.5.2 Core Functionalities

Registration and login; equipment donation, browsing and details; borrow request creation and processing; return initiation with condition assessment; inspection; sanitization tracking; notifications; messaging; user, equipment, request and return management; and analytics and reports.

### 1.5.3 Technical Scope

The scope covers a React/TypeScript single-page client with 27 pages, 7 reusable components, 3 layouts and 2 context providers; an Express/TypeScript REST API with 8 route modules, 8 controllers and 3 middleware modules; and a MongoDB database with 6 collections and a seeding utility that loads demonstration data. Together the source code comprises approximately 9,700 lines of TypeScript.

### 1.5.4 Limitations

The project does not process payments, does not provide courier integration, does not verify donor identity documents or medical prescriptions, and does not yet provide automated test coverage. Image upload middleware exists on the server but is not connected to any route; images are supplied as URLs.

### 1.5.5 Future Scope

Future work includes cloud storage, real-time messaging, push/e-mail/SMS notifications, geo-location search, mobile applications, multilingual interfaces, analytics enhancements, automated testing and deployment pipelines, and a stricter security model (see Section 8.2).

# Chapter 2: Literature Review

## 2.1 Evolution

The reuse of medical equipment is an old practice: charitable “loan closets”, hospital donation drives and community organizations have long lent crutches, wheelchairs and beds to people in need. Traditionally these services were run from a physical store room, with registers maintained by volunteers. Their reach was limited to a single locality, availability information was known only to the volunteers, and the condition and hygiene of items were rarely documented.

The growth of the internet and of the *collaborative consumption* movement [1] changed the way underused goods are exchanged. Online classified services and peer-to-peer sharing platforms made it possible to advertise a single item to an entire city. However, these general-purpose platforms were not designed for medical equipment and provide no verification, no hygiene tracking and no structured return process.

At the same time, public-health organizations have emphasized that access to assistive products remains very uneven and that appropriate reuse and quality assurance are important parts of making such products available [2]. The World Health Organization also publishes guidance on the donation of medical devices, which stresses that donated equipment must be appropriate, usable and properly supported to be of real benefit [3]. These observations motivate a platform in which donations are moderated and condition is explicitly recorded.

On the technology side, the maturity of JavaScript/TypeScript full-stack development has made it practical to build such platforms quickly. REST [4] became the dominant style for web APIs; JSON Web Tokens [5] provide stateless authentication; adaptive password hashing such as bcrypt [6] protects stored credentials; and document databases such as MongoDB [7] allow flexible schemas for items with varied specifications. Component-based user-interface libraries [8] and utility-first CSS frameworks [9] allow responsive interfaces to be assembled consistently.

## 2.2 Existing Platforms

Existing solutions for obtaining medical equipment fall into the following broad categories. The comparison in Table 2.1 is a qualitative one, based on the typical characteristics of each category rather than on any single named product.

**Table 2.1: Qualitative comparison of existing approaches with CareShare**

| Approach | Strengths | Weaknesses | CareShare response |
|---|---|---|---|
| Charitable loan closets / NGO banks | Trusted, low cost, community-driven | Local only; manual registers; little online visibility; hygiene rarely documented | Online catalogue with verification and sanitization status |
| General classified / resale websites | Wide reach; easy listing | No verification; sale-oriented; no return or damage process; trust issues | Moderated listings, structured borrow and return lifecycle |
| Rental / commercial suppliers | Professional service; maintained equipment | Cost to the patient; limited range; geographic limits | Free community lending between donors and borrowers |
| Hospital or government schemes | Subsidised; authoritative | Eligibility rules; paperwork; limited stock | Open platform that complements formal schemes |
| Social media groups / messaging apps | Fast, informal | No search, no audit trail, privacy risks | Searchable catalogue, tracked status history, controlled messaging |

From this comparison it can be concluded that there is room for a dedicated, moderated, free-of-charge platform that combines the community spirit of loan closets with the reach and traceability of web applications. CareShare is designed to occupy this space.

## 2.3 Summary of Literature Review

The literature and the survey of existing approaches lead to the following observations that shaped the design of CareShare:

- Reuse of medical equipment is socially and economically valuable, but only when equipment is appropriate, clean and in working condition [2], [3].
- Informal channels lack moderation and an audit trail, so a verification step and a stored status history are essential.
- Stateless token-based authentication [5] with salted adaptive hashing [6] is a suitable, scalable choice for a REST API [4].
- Access-control failures are among the most common and most serious web-application weaknesses [10]; therefore authorization must be enforced on the server for every sensitive operation and not only in the user interface.
- A document database [7] suits equipment records whose specifications vary between categories.

# Chapter 3: System Requirement Study

## 3.1 Study of Current System

In the current (manual) situation, a family that needs a wheelchair or a patient bed for a short duration typically relies on personal contacts, social-media groups, a local charitable store or a commercial rental. A donor with spare equipment has no reliable way to find a recipient and often discards the item or leaves it unused. Information about availability, condition, sanitization and return dates is exchanged verbally or through messaging applications and is not recorded in any structured form.

## 3.2 Problems and Weaknesses of Current System

- **Poor discoverability** – equipment can be found only through personal networks or a single local store.
- **No verification** – there is no independent check of equipment condition or hygiene before it is lent.
- **No traceability** – requests, hand-overs and returns are not recorded; disputes about damage cannot be resolved.
- **Hygiene risk** – sanitization after use is not documented.
- **Inefficient coordination** – donors and borrowers must exchange phone numbers and messages without context.
- **No analytics** – organizations running an equipment bank cannot measure usage, demand or damage rates.

## 3.3 User Characteristics

**Table 3.1: User classes and their characteristics**

| User class | Typical profile | Technical skill | Main activities |
|---|---|---|---|
| Guest | Anyone visiting the site | Basic | Browse catalogue, read information pages, register |
| Borrower | Patient, caregiver or family member; often under stress and time pressure | Basic to moderate; frequently on a mobile phone | Search equipment, submit requests, track status, return equipment, message donors |
| Donor | Household or organization with surplus equipment | Basic to moderate | List equipment, respond to requests, inspect returns |

| User class | Typical profile | Technical skill | Main activities |
|---|---|---|---|
| Dual-role user | Individual who lends and borrows | Basic to moderate | All donor and borrower activities |
| Administrator | Bank staff or volunteer coordinator | Moderate | Verify listings, record sanitization, manage users, review reports |

## 3.4 Hardware and Software Requirements

### Hardware Requirements

**Table 3.2: Hardware requirements**

| Component | Development / server | Client (end user) |
|---|---|---|
| Processor | Dual-core 2.0 GHz or better | Any processor capable of running a modern browser |
| Memory (RAM) | 4 GB minimum, 8 GB recommended | 2 GB or more |
| Storage | 2 GB free (dependencies, build output, database) | Not applicable |
| Network | Broadband connection for MongoDB Atlas and package installation | Any stable internet connection |
| Display | Any | Responsive layout, from smartphone to desktop |

### Software Requirements

**Table 3.3: Software requirements**

| Category | Requirement |
|---|---|
| Operating system | Windows, Linux or macOS (the project was developed on Windows) |
| Runtime | Node.js (LTS) with npm |
| Server language / framework | TypeScript 5.5, Express 4.19 |
| Client language / framework | TypeScript 5.5, React 18.3, Vite 5.3, Tailwind CSS 3.4 |
| Database | MongoDB (Atlas cloud cluster or local instance), Mongoose 8.5 |
| Browser | Current versions of Chrome, Edge, Firefox or Safari |
| Tools | Visual Studio Code, Git, a REST client such as Postman (optional) |

## 3.5 Constraints

### 3.5.1 User Interface

The interface must be responsive, readable and usable by non-technical users, including people using a phone. It follows a consistent blue/teal medical colour scheme, uses clear status badges and shows loading skeletons and toast messages for feedback.

### 3.5.2 Communications Interface

Client and server communicate over HTTP(S) using JSON. During development the Vite dev server (port 5173) proxies /api calls to the Express server (port 5000). Authentication uses the Authorization: Bearer <token> header. Cross-origin requests are restricted to the configured CLIENT_URL.

### 3.5.3 Hardware Interface

The application has no direct hardware interface. It relies on the standard input devices and display of the user's computer or phone. Photographs of equipment are supplied by the user as image URLs.

### 3.5.4 Criticality of the Application

CareShare is not a life-critical system; it does not control medical devices or store clinical records. However, it deals with equipment used by patients and with personal contact data, so integrity of listings, correctness of hygiene information and confidentiality of user data are important. Availability matters mainly because recovering patients often need equipment urgently.

### 3.5.5 Safety and Security Considerations

- Passwords are stored only as bcrypt hashes; the password field is excluded from user queries that populate the request context.
- Protected endpoints require a valid JWT; suspended users are rejected at the middleware and at login.
- Helmet sets standard security headers, CORS is restricted to the client origin and a rate limiter allows at most 300 requests per 15 minutes per IP address on /api.
- New equipment is not public until verified by an administrator; sanitization status is shown on every item.

- Registration input is validated with Zod; Mongoose schemas enforce enumerations and required fields.

## 3.6 Assumptions and Dependencies

### 3.6.1 Assumptions

- Users have access to a modern browser and an internet connection.
- Donors provide truthful information about equipment, and administrators perform the verification honestly and promptly.
- Pickup, delivery and hand-over are arranged directly between donor and borrower outside the system.
- A MongoDB instance is reachable; if not, the server falls back to a temporary in-memory database for demonstration purposes.

### 3.6.2 Dependencies

The system depends on the open-source packages listed in the package.json files of the client and server (Chapter 5), on the Node.js runtime, on MongoDB, and, for demonstration images, on externally hosted image URLs. The in-memory database fallback additionally requires the mongodb-memory-server package (see Section 7.5).

# Chapter 4: System Design

## 4.1 Overview of System Workflow

CareShare follows a three-tier client–server architecture (Figure 4.1). The React client renders pages, keeps the authenticated user in a context provider and calls the REST API through a single Axios instance that automatically attaches the JWT. The Express server applies security middleware, authenticates the caller, routes the request to a controller and applies the business rules. Controllers use Mongoose models to read and write MongoDB collections.

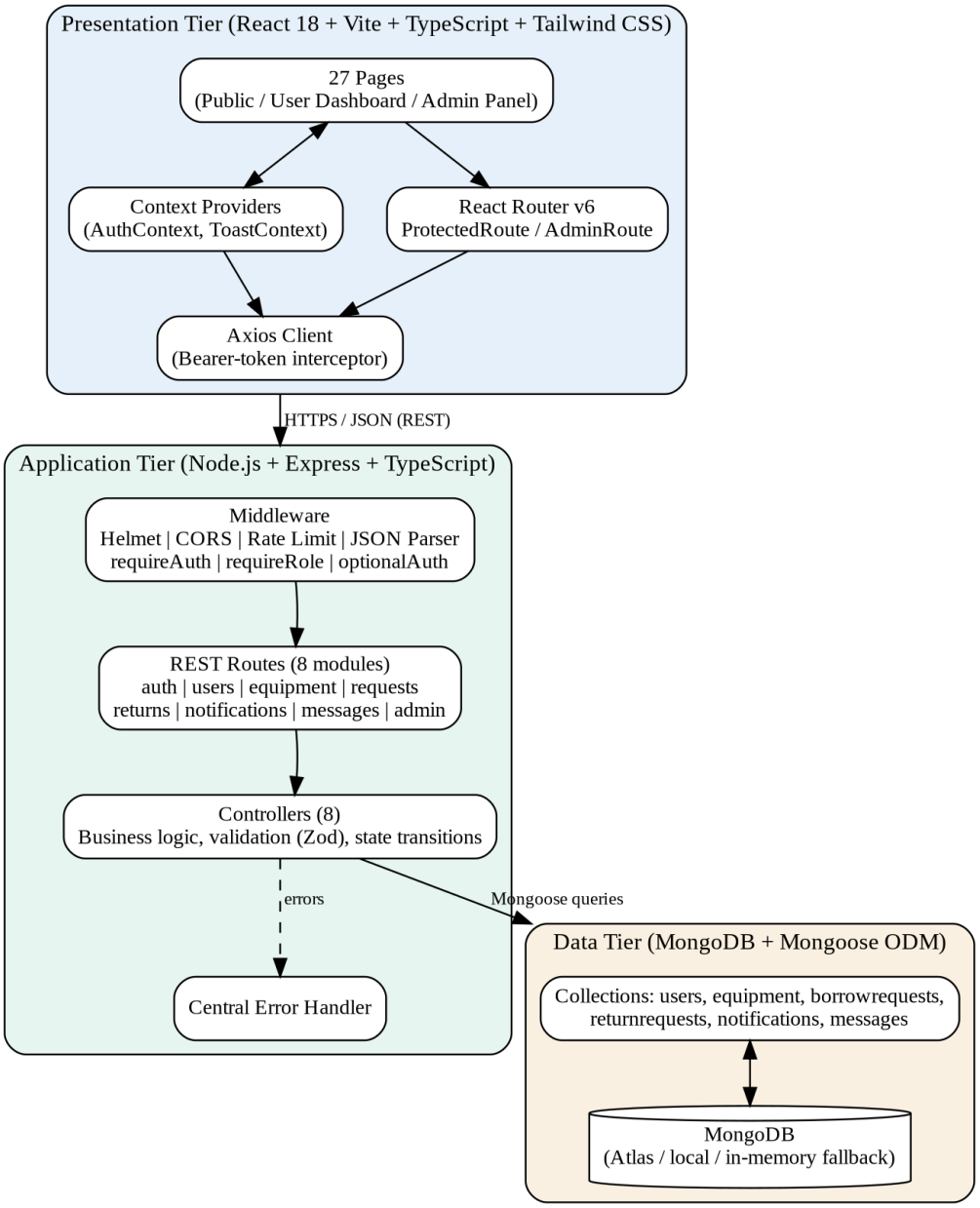



**Figure 4.1: Three-tier architecture of CareShare**

The main business workflow is shown in Figure 4.2. A donor submits equipment, which an administrator verifies. Approved equipment becomes available in the public catalogue. A borrower submits a request, the donor approves or rejects it, and after hand-over the borrower initiates a return. The donor or an administrator inspects the item; a passed inspection completes the request, returns the item to the catalogue and flags it for sanitization, while a damaged item is made unavailable.

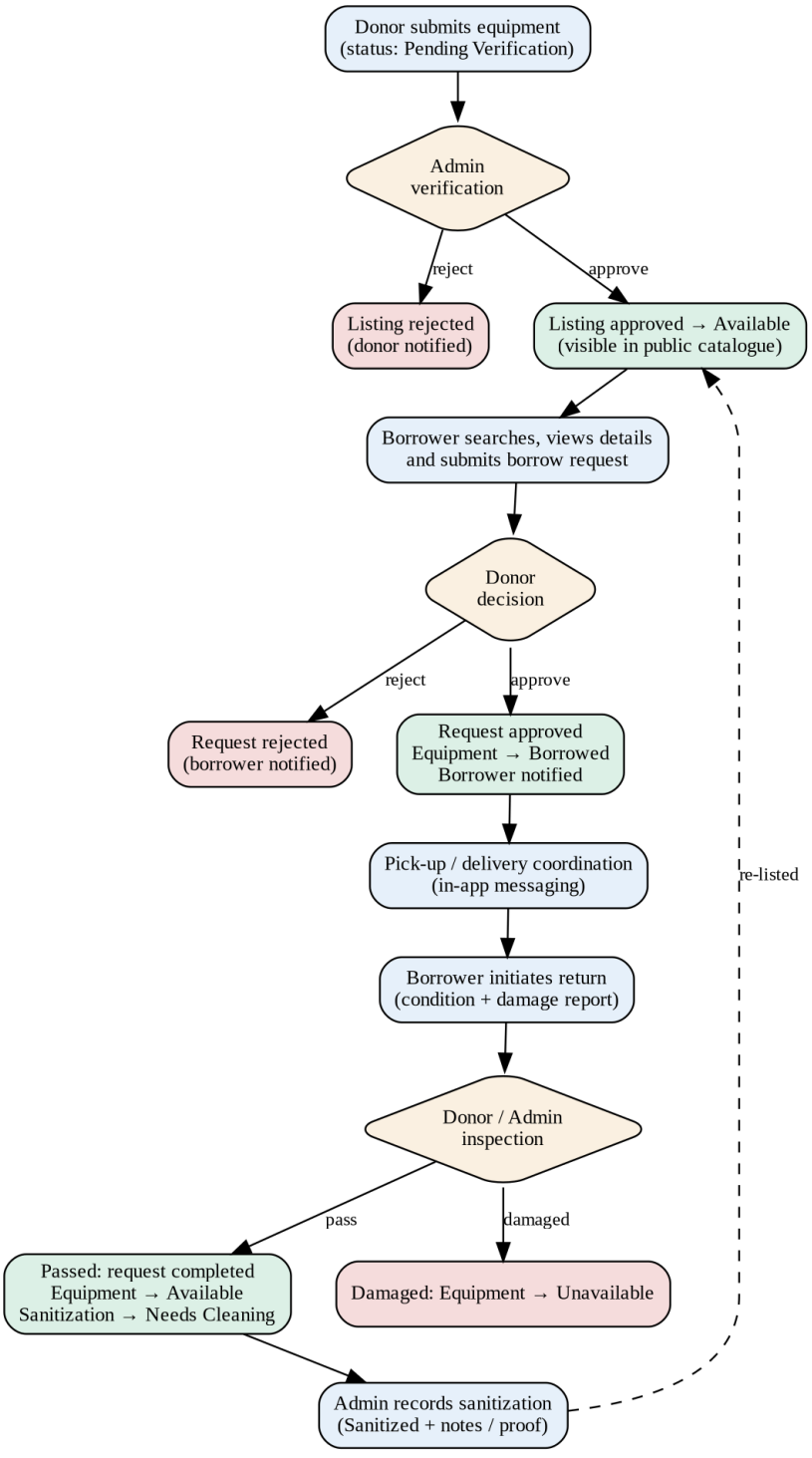



**Figure 4.2: End-to-end equipment sharing workflow**

The borrow request itself is a finite-state machine. The server defines the permitted transitions in a lookup table (VALID_TRANSITIONS), and every change is appended to the statusHistory array together with the user who made it and an optional note. Figure 4.3 shows the states and transitions.

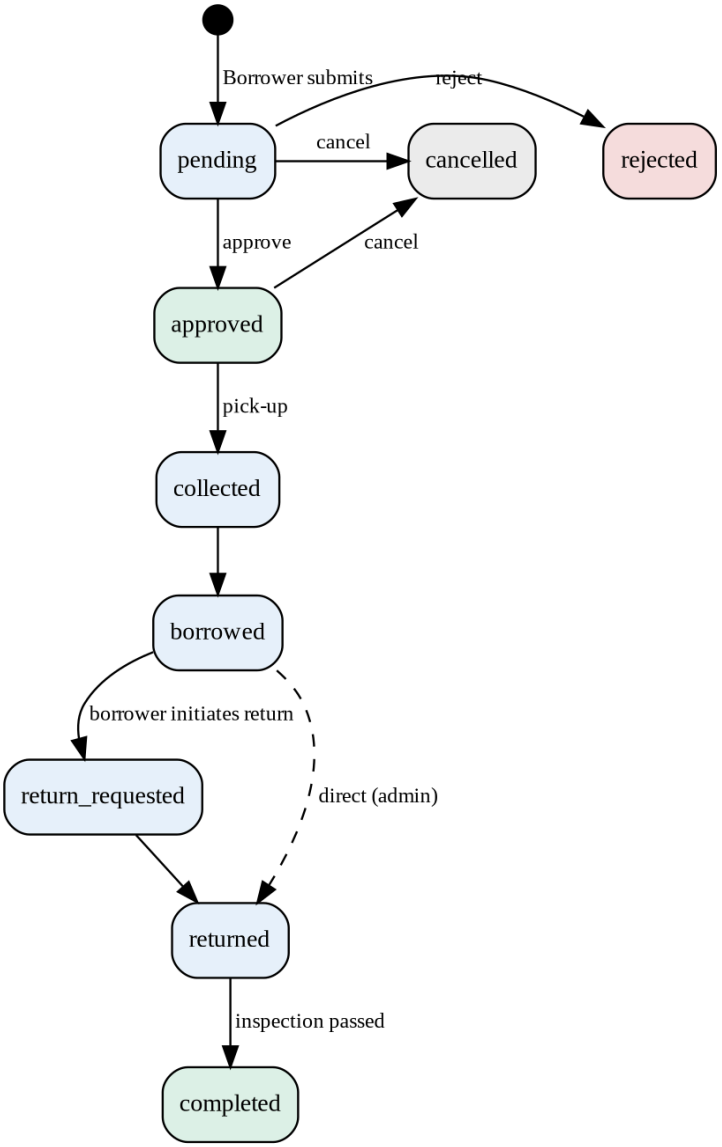



**Figure 4.3: State diagram of a borrow request**

Figure 4.4 shows the use cases supported for each actor.

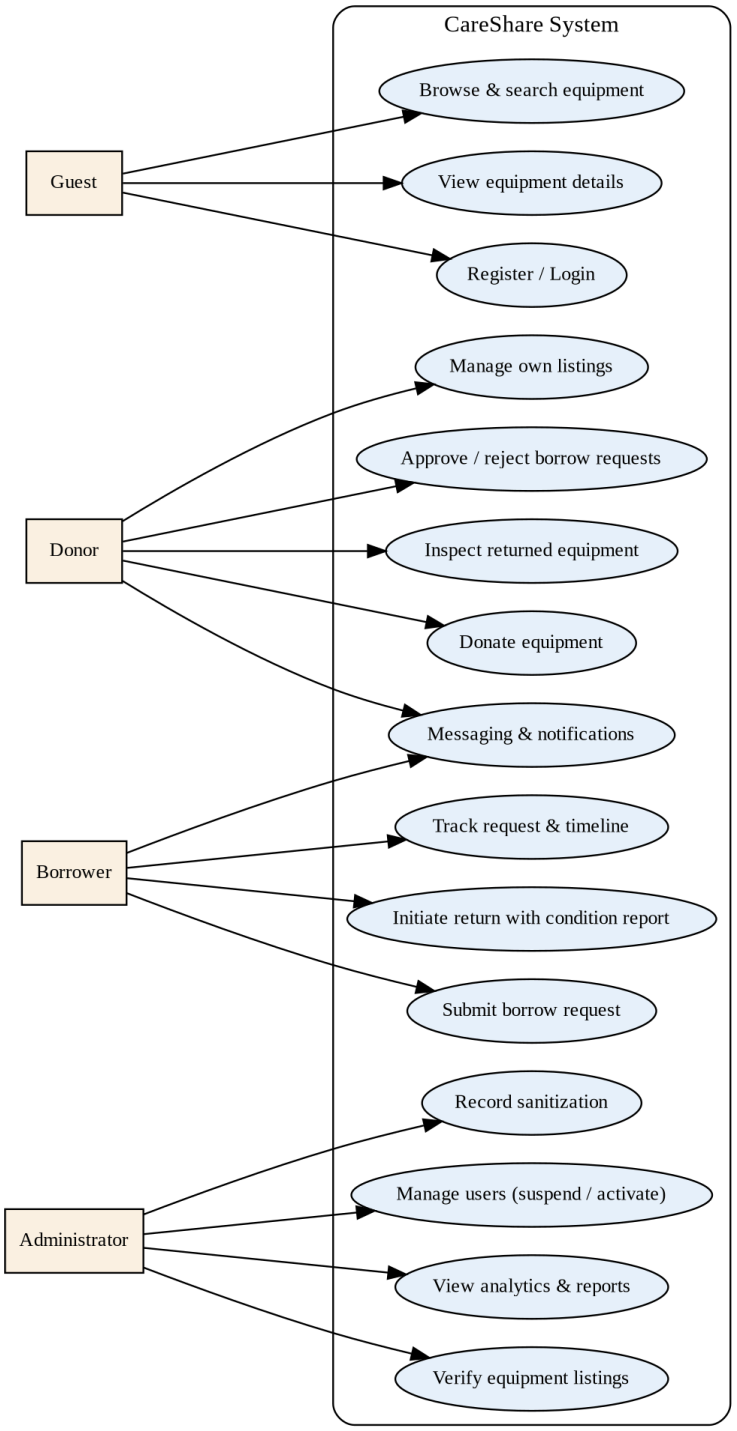



**Figure 4.4: Use-case diagram**

## 4.2 Database Design and ER Diagrams

### 4.2.1. Database Structure

The database is a MongoDB database named CareShare. It contains six collections, each defined by a Mongoose schema with timestamps (createdAt, updatedAt). Relationships

between documents are expressed by ObjectId references and resolved with populate(). Tables 4.1 to 4.6 describe the collections.

**Table 4.1: Collection: users**

| Field | Type | Constraints / description |
|---|---|---|
| fullName | String | Required, trimmed |
| email | String | Required, unique, lower-cased, indexed |
| phone | String | Required |
| password | String | Required; stores the bcrypt hash only |
| address, city | String | Required; city is indexed |
| role | String (enum) | borrower \| donor \| both \| admin; default borrower; indexed |
| avatar | String | Optional image URL |
| isSuspended | Boolean | Default false; blocks login and API access |

**Table 4.2: Collection: equipment**

| Field | Type | Constraints / description |
|---|---|---|
| name | String | Required, trimmed, indexed |
| category | String (enum) | Wheelchairs, Walkers, Crutches, Patient Beds, Oxygen Stands, Commode Chairs, Bedside Tables, Walking Frames; indexed |
| description | String | Required |
| condition | String (enum) | Like New \| Good \| Fair \| Refurbished |
| purchaseYear | Number | Optional |
| images | String[] | Image URLs |
| owner | ObjectId → User | Required, indexed |
| location | Embedded object | city (required, indexed), area, address (required), pincode |
| availabilityStatus | String (enum) | Available \| Borrowed \| Pending Verification \| Rejected \| Unavailable; default Pending Verification |
| verificationStatus | String (enum) | pending \| approved \| rejected; default pending |
| sanitizationStatus | String (enum) | Pending \| Sanitized \| Needs Cleaning; default Pending |
| lastSanitizedAt, sanitizedBy, sanitizationNotes, sanitizationProof | Date / String | Sanitization record |
| verifiedBy, verifiedAt, verificationNotes | ObjectId / Date / String | Verification record |

| Field | Type | Constraints / description |
|---|---|---|
| specifications | Map<String,String> | Free-form technical specifications |
| viewCount, borrowCount | Number | Popularity counters, default 0 |

**Table 4.3: Collection: borrowrequests**

| Field | Type | Constraints / description |
|---|---|---|
| equipment, borrower, donor | ObjectId → Equipment / User / User | Required, each indexed |
| purpose | String | Required |
| startDate, expectedReturnDate | Date | Required |
| actualReturnDate | Date | Set when the item is returned/completed |
| quantity | Number | Default 1 |
| pickupPreference | String (enum) | Self Pickup \| Home Delivery \| Community Hub Pickup |
| address, notes, donorNotes | String | address is required |
| status | String (enum) | Nine states (Figure 4.3); default pending; indexed |
| statusHistory | Array of sub-documents | status, updatedBy (User), updatedAt, note |

**Table 4.4: Collection: returnrequests**

| Field | Type | Constraints / description |
|---|---|---|
| borrowRequest, equipment, borrower, donor | ObjectId references | Required, each indexed |
| returnDate | Date | Default now |
| conditionAfterUse | String (enum) | Excellent \| Good \| Fair \| Damaged; default Good |
| damageReport, notes | String | Free text |
| photos | String[] | Evidence photo URLs |
| inspectionStatus | String (enum) | Passed \| Needs Inspection \| Damaged; default Needs Inspection; indexed |
| inspectionNotes, inspectedBy, inspectedAt | String / ObjectId / Date | Inspection record |
| isCompleted | Boolean | True when inspection passed |

**Table 4.5: Collection: notifications**

| Field | Type | Constraints / description |
|---|---|---|
| user | ObjectId → User | Required, indexed (recipient) |
| type | String (enum) | request_submitted, request_approved, request_rejected, equipment_verified, equipment_rejected, return_due, return_requested, equipment_returned, system |
| title, message | String | Required |
| link | String | Client route opened from the notification |
| read | Boolean | Default false, indexed |

**Table 4.6: Collection: messages**

| Field | Type | Constraints / description |
|---|---|---|
| conversationId | String | Required, indexed; the two user ids sorted and joined by "_" |
| sender, recipient | ObjectId → User | Required, indexed |
| equipment | ObjectId → Equipment | Optional context |
| text | String | Required, trimmed |
| read | Boolean | Default false |

### 4.2.2. Entity-Relationship (ER) Model

Figure 4.5 shows the entity-relationship model. A user owns many pieces of equipment (one-to-many). A piece of equipment can be the subject of many borrow requests, and each borrow request links exactly one equipment item, one borrower and one donor. A borrow request may have zero or more return records. Users receive many notifications and exchange many messages; a message can optionally refer to an equipment item.

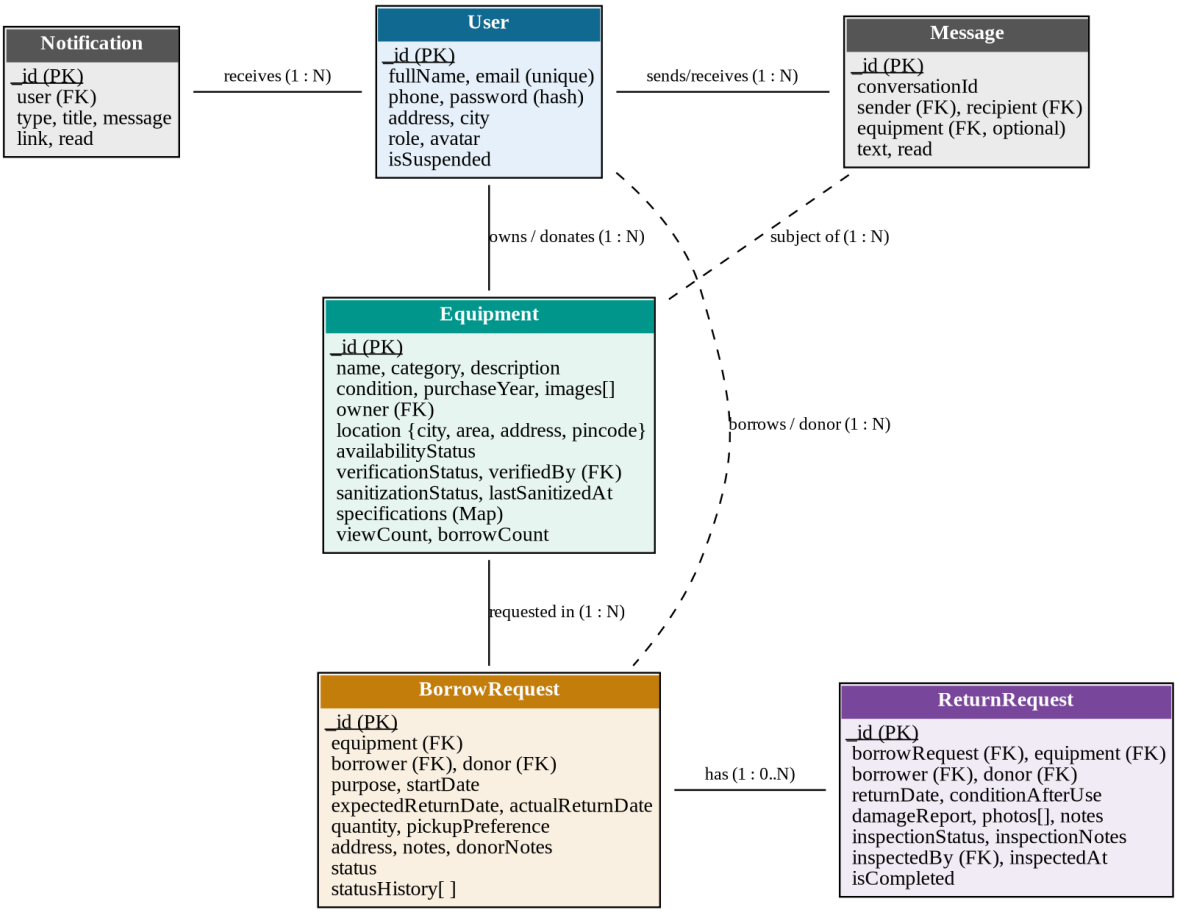



**Figure 4.5: Entity-relationship diagram of CareShare**

## 4.3 User Interface and Experience Design (UI/UX)

### 4.3.1. User Interface Design Principles

The interface is built from reusable components styled with Tailwind CSS utility classes. The design follows these principles:

- **Consistent medical palette** – deep navy (#0B3558) for headings and navigation, teal (#00AFA3) for actions and highlights, light blue and mint backgrounds, defined once in tailwind.config.js.
- **Readable typography** – the Inter typeface with clear weight hierarchy.
- **Responsive layout** – grid and flex utilities adapt cards, forms and tables from phones to desktops.
- **Status visibility** – the StatusBadge component renders request, verification, sanitization and availability states with colour and text; RequestTimeline shows the progress of a request.

- **Three layouts** – MainLayout (public), DashboardLayout (signed-in users) and AdminLayout (administrators) share navigation patterns but expose different menus.

Table 4.7 lists the application routes and the layout in which each is displayed.

**Table 4.7: Application routes and access level**

| Area | Routes | Access |
|---|---|---|
| Public | /, /about, /how-it-works, /equipment, /equipment/:id, /donate, /contact, /faq, /login, /register | Everyone (donate and request actions require sign-in) |
| User dashboard | /dashboard, /my-equipment, /my-requests, /borrowed, /returns, /notifications, /messages, /profile, /settings | Signed-in users (ProtectedRoute) |
| Admin panel | /admin, /admin/users, /admin/equipment, /admin/verification, /admin/sanitization, /admin/requests, /admin/returns, /admin/reports | Role “admin” only (AdminRoute) |

### 4.3.2. User Experience Enhancements

- **Loading skeletons** (LoadingSkeleton) avoid blank screens while data loads.
- **Toast notifications** (ToastContext) confirm success or explain errors without leaving the page.
- **Quick-fill demo logins** on the sign-in page allow evaluators to try every role in one click.
- **Request modal** on the equipment page pre-fills dates (today and today + 14 days), the user’s address and a default pickup preference.
- **Dashboards** summarise active requests, borrowed items, donations and notifications for the user, and platform statistics for the administrator.
- **Guided information pages** (How It Works, FAQ, About, Contact) explain the process to first-time users.

## 4.4 Class Diagrams and System Components

### 4.4.1. Major System Components

**Table 4.8: Major system components**

| Component | Location | Responsibility |
|---|---|---|
| Express application | server/src/app.ts | Security headers, CORS, rate limiting, body parsing, static uploads, route mounting, health check, error handler |

| Component | Location | Responsibility |
|---|---|---|
| Server entry point | server/src/server.ts | Connects to the database and starts listening (default port 5000) |
| Database configuration | server/src/config/db.ts | Connects to MongoDB; falls back to an in-memory server; triggers auto-seeding |
| Models (6) | server/src/models | Mongoose schemas for User, Equipment, BorrowRequest, ReturnRequest, Notification, Message |
| Controllers (8) | server/src/controllers | Business logic for auth, users, equipment, requests, returns, notifications, messages and admin |
| Routes (8) | server/src/routes | Map HTTP verbs and paths to controllers and attach middleware |
| Middleware (3) | server/src/middleware | Authentication and role checks, error handling, Multer upload configuration |
| Seed utilities | server/src/seed | Create demonstration users, equipment, request, notifications and messages |
| Client application | client/src | Routing (App.tsx), layouts, 27 pages, 7 components, context providers, API client, shared types |

### 4.4.2. Class Diagram Structure

Figure 4.6 shows the data classes (the Mongoose models) and their associations. Figure 4.7 shows the controller and middleware classes of the server and their use of the models.

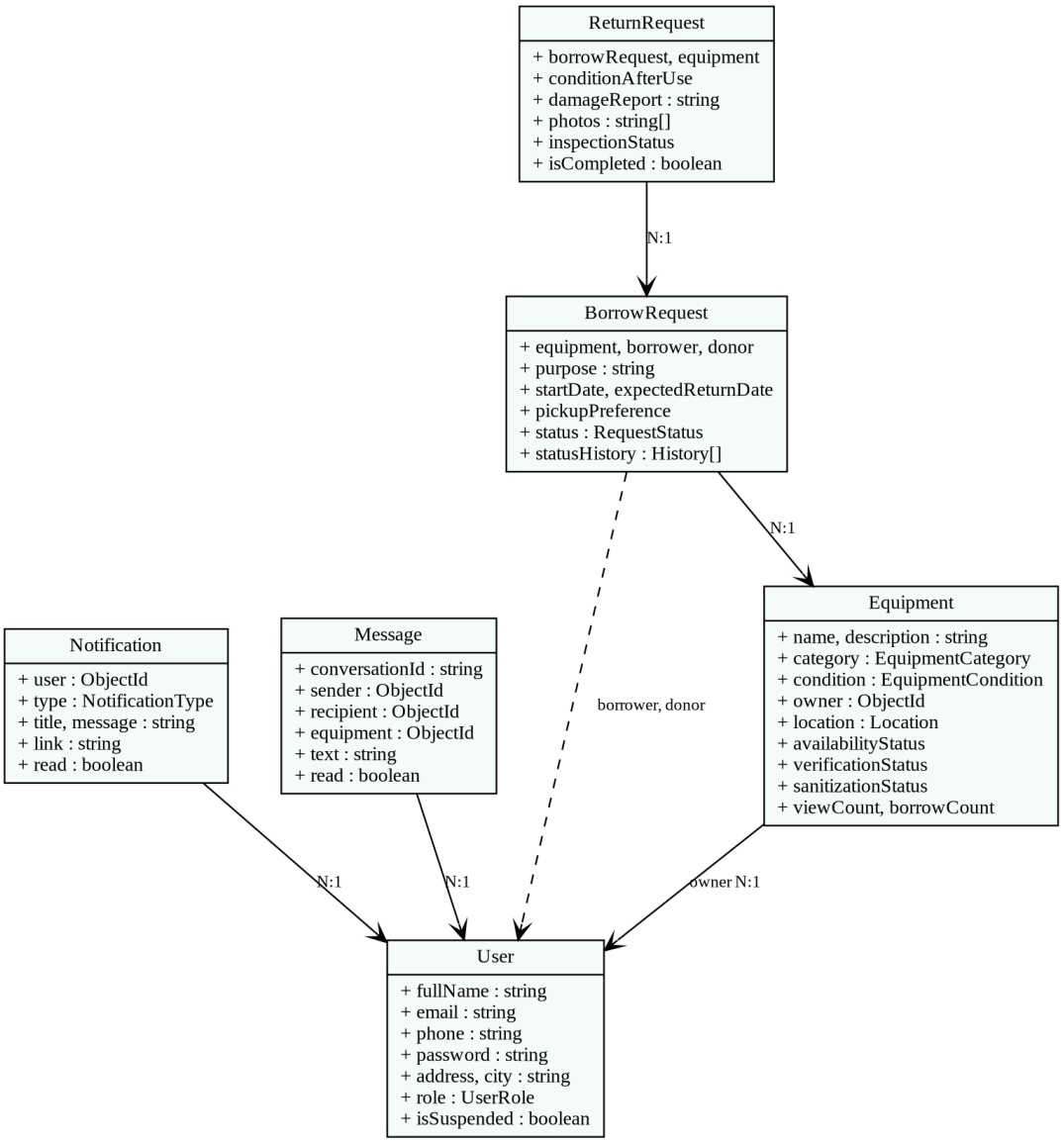



**Figure 4.6: Class diagram of the data models**

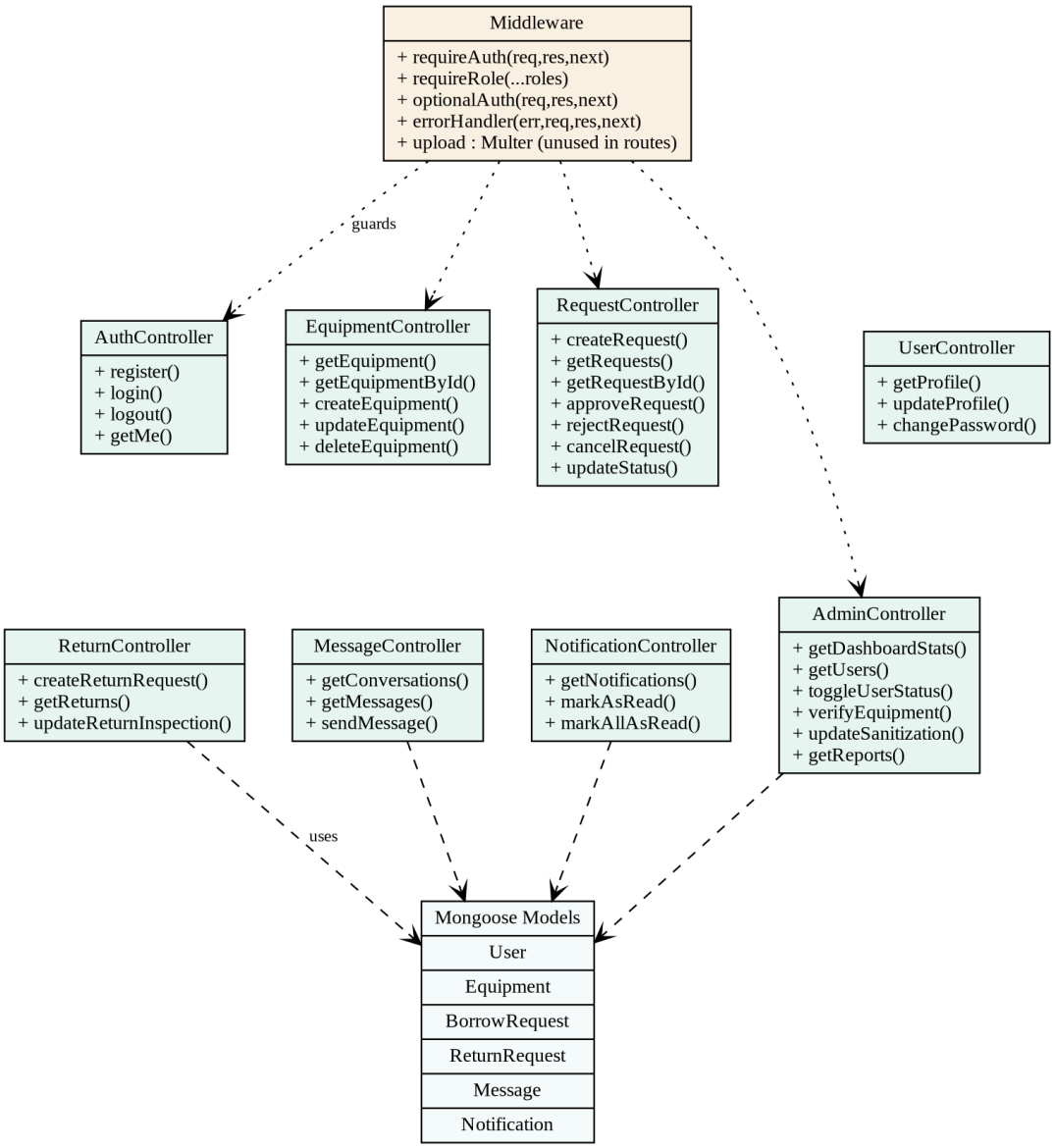



**Figure 4.7: Class diagram of controllers and middleware**

# Chapter 5: Implementation and Development

This chapter describes how the design was implemented. Code excerpts are taken from the project source.

## 5.1 Technology Stack

**Table 5.1: Technology stack and versions (from package.json)**

| Layer | Technology | Version | Purpose |
|---|---|---|---|
| Client | React / React DOM | ^18.3.1 | Component-based user interface |
| Client | TypeScript | ^5.5.3 | Static typing |
| Client | Vite | ^5.3.3 | Dev server with API proxy and production build |
| Client | Tailwind CSS | ^3.4.4 | Utility-first styling |
| Client | React Router DOM | ^6.24.1 | Client-side routing and guards |
| Client | Axios | ^1.7.2 | HTTP client with interceptors |
| Client | TanStack Query | ^5.51.1 | Query client provider (configured in main.tsx) |
| Client | Lucide React | ^0.400.0 | Icon set |
| Server | Node.js + Express | ^4.19.2 | HTTP server and routing |
| Server | Mongoose | ^8.5.1 | MongoDB object-data mapping |
| Server | jsonwebtoken | ^9.0.2 | JWT signing and verification |
| Server | bcryptjs | ^2.4.3 | Password hashing (cost factor 10) |
| Server | Zod | ^3.23.8 | Registration input validation |
| Server | Helmet, CORS, express-rate-limit | ^7.1.0, ^2.8.5, ^7.3.1 | Security headers, origin control, request throttling |
| Server | Multer | ^1.4.5-lts.1 | Multipart upload handling (image type and 5 MB limit) |
| Server | tsx, dotenv | ^4.16.2, ^16.4.5 | TypeScript execution in development; environment variables |
| Database | MongoDB | Atlas / local | Document database |

## 5.2 Frontend Implementation

**Entry point and providers.** main.tsx wraps the application with QueryClientProvider, BrowserRouter, AuthProvider and ToastProvider. App.tsx declares all routes in three groups (public, authenticated and admin) and defines two guard components.

```
const ProtectedRoute = ({ children }) => {
```

```
  const { user } = useAuth();
  if (!user) return <Navigate to="/login" replace />;
  return <>{children}</>;
};

const AdminRoute = ({ children }) => {
  const { user } = useAuth();
  if (!user || user.role !== 'admin') return <Navigate to="/dashboard" replace />;
  return <>{children}</>;
};
```

**Listing 5.1: Route guards in App.tsx**

**Authentication state.** AuthContext keeps the user and token in React state and mirrors them to localStorage (careshare_token, careshare_user). On start-up it calls GET /auth/me to refresh the user from the server. It exposes login, logout, updateUser and quickLogin; the latter signs in with the demonstration accounts and, if the backend is unreachable, creates a local demo session.

**API client.** A single Axios instance with base URL /api adds the bearer token in a request interceptor and clears the stored session when a 401 response is received on dashboard or admin routes.

```
api.interceptors.request.use((config) => {
  const token = localStorage.getItem('careshare_token');
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});
```

**Listing 5.2: Token injection in api/client.ts**

**Pages.** The 27 pages are grouped as follows: 10 public pages (Home, About, How It Works, Browse Equipment, Equipment Details, Donate Equipment, Contact, FAQ, Login, Register), 9 user pages (Dashboard, My Equipment, My Requests, Borrowed Equipment, Returns, Notifications, Messages, Profile, Settings) and 8 admin pages (Dashboard, Users, Equipment, Verification, Sanitization, Requests, Returns, Reports). Each page fetches data with Axios inside useEffect, keeps loading and form state with useState, and reports results through toasts.

**Components.** Navbar and Footer provide navigation; EquipmentCard displays a catalogue item; StatusBadge and RequestTimeline display state; StatsCard shows dashboard figures; LoadingSkeleton provides placeholders. TypeScript interfaces in types/index.ts mirror the server models so that responses are type-checked in the client.

**Donation form.** DonateEquipment collects name, category, description, condition, purchase year, image URL, location, sanitization status/notes and a multi-line specification list ("key: value" per line) which is converted to an object before the POST request to /equipment.

**Admin reports.** AdminReports builds a CSV string from the report data and downloads it through a data URI, providing exportable reports without an additional library.

## 5.3 Backend Implementation

**Application set-up.** app.ts registers Helmet (with the content-security policy disabled), CORS restricted to CLIENT_URL, a rate limiter on /api (300 requests per 15 minutes), JSON and URL-encoded body parsers (10 MB limit), a static /uploads directory, the eight route modules, a health endpoint and the central error handler.

```
app.use(helmet({ contentSecurityPolicy: false }));
app.use(cors({ origin: process.env.CLIENT_URL || 'http://localhost:5173',
               credentials: true }));
app.use('/api', rateLimit({ windowMs: 15 * 60 * 1000, max: 300 }));
app.use(express.json({ limit: '10mb' }));
app.use('/api/auth', authRoutes);   // ... users, equipment, requests,
                                    // returns, notifications, messages, admin
```

**Listing 5.3: Middleware and route mounting in app.ts**

**REST API.** Table 5.2 lists all endpoints. Every route that changes data requires authentication.

**Table 5.2: REST API endpoints**

| Method and path | Auth / role | Description |
|---|---|---|
| POST /api/auth/register | Public | Validate with Zod, hash password, create user, return JWT |
| POST /api/auth/login | Public | Verify credentials, reject suspended users, return JWT |
| POST /api/auth/logout | Public | Acknowledges logout (token is discarded by the client) |
| GET /api/auth/me | Authenticated | Return current user |
| GET /api/users/profile | Authenticated | Return own profile |
| PUT /api/users/profile | Authenticated | Update name, phone, address, city, avatar |
| PUT /api/users/password | Authenticated | Change password after checking the current one |
| GET /api/equipment | Optional | Search, filter, sort and paginate; only approved items unless admin |

| Method and path | Auth / role | Description |
|---|---|---|
| GET /api/equipment/:id | Optional | Equipment details; increments view counter |
| POST /api/equipment | donor, both, admin | Create listing in Pending Verification state |
| PUT /api/equipment/:id | Owner or admin | Update listing |
| DELETE /api/equipment/:id | Owner or admin | Delete listing |
| POST /api/requests | Authenticated | Create borrow request for an available, approved item |
| GET /api/requests | Authenticated | List own requests as borrower or donor (all for admin) |
| GET /api/requests/:id | Authenticated | Request details with status history |
| PUT /api/requests/:id/approve | Donor of request or admin | Approve; equipment becomes Borrowed; notify borrower |
| PUT /api/requests/:id/reject | Donor of request or admin | Reject with optional reason; notify borrower |
| PUT /api/requests/:id/cancel | Borrower or admin | Cancel request |
| PUT /api/requests/:id/status | Authenticated | Advance status according to the transition table |
| POST /api/returns | Borrower or admin | Create return record; request becomes return_requested |
| GET /api/returns | Authenticated | Own returns (all for admin) |
| PUT /api/returns/:id/inspection | Donor of return or admin | Record inspection result; update equipment and request |
| GET /api/notifications | Authenticated | Latest 50 notifications and unread count |
| PUT /api/notifications/read-all, /:id/read | Authenticated | Mark notifications as read |
| GET /api/messages/conversations | Authenticated | List conversations |
| GET /api/messages/messages/:conversationId | Authenticated | List messages and mark them as read |
| POST /api/messages/messages | Authenticated | Send a message |
| GET /api/admin/dashboard | admin | Platform statistics |
| GET /api/admin/users | admin | List and filter users |
| PUT /api/admin/users/:id/toggle-status | admin | Suspend or activate (non-admin) user |
| PUT /api/admin/equipment/:id/verify | admin | Approve or reject a listing |
| PUT /api/admin/equipment/:id/sanitization | admin | Record sanitization |
| GET /api/admin/reports | admin | Report metrics and category statistics |

| Method and path | Auth / role | Description |
|---|---|---|
| GET /api/health | Public | Health check |

**Authentication controller.** Registration input is validated with a Zod schema (name of at least 2 characters, valid e-mail, phone of at least 10 characters, password of at least 6 characters, matching confirmation, address, city and role). The password is hashed with bcrypt (cost factor 10) before the user is stored and a JWT valid for 7 days is returned.

```
const hashedPassword = await bcrypt.hash(password, 10);
const user = await User.create({ fullName, email: email.toLowerCase(),
  phone, password: hashedPassword, address, city, role });
const token = jwt.sign({ id: userId }, secret, { expiresIn: '7d' });
```

**Listing 5.4: Registration in authController.ts**

**Request controller.** createRequest checks that the equipment is approved and available, that the requester is not the owner and that the requester has no active request for the same item, then creates the request with its first history entry and notifies the donor. approveRequest marks the equipment as Borrowed and increments its borrow counter; updateStatus validates transitions against VALID_TRANSITIONS; when a request becomes returned or completed the equipment is set back to Available.

```
const VALID_TRANSITIONS: Record<RequestStatus, RequestStatus[]> = {
  pending: ['approved', 'rejected', 'cancelled'],
  approved: ['collected', 'cancelled'],
  rejected: [], cancelled: [],
  collected: ['borrowed'],
  borrowed: ['return_requested', 'returned'],
  return_requested: ['returned'],
  returned: ['completed'],
  completed: []
};
```

**Listing 5.5: Request state-transition table**

**Return controller.** createReturnRequest stores the condition report and sets the request to return_requested. updateReturnInspection records the inspector, and, if the result is Passed, completes the borrow request, sets the equipment to Available and marks its sanitization status as Needs Cleaning; if the result is Damaged the equipment becomes Unavailable.

**Admin controller.** getDashboardStats aggregates user, equipment, request and category counts. verifyEquipment sets verification and availability status and notifies the donor; updateSanitization stores status, notes, proof reference and timestamp; toggleUserStatus

suspends or reactivates non-admin users; getReports returns verification and completion rates and damaged-return counts.

**Error handling.** Controllers wrap their logic in try/catch blocks and return a uniform JSON structure { success, message, data }. A central errorHandler middleware logs unexpected errors and includes the stack trace only when NODE_ENV is development.

## 5.4 Database Integration

config/db.ts reads MONGODB_URI from the environment and connects with Mongoose. A short server-selection timeout (2 seconds) allows the server to detect an unreachable database quickly; in that case it starts a temporary in-memory MongoDB instance so that the application can still be demonstrated. After connecting, seedInitialData() checks whether the users collection is empty and, if so, creates demonstration data: four users (admin, donor, borrower and dual-role), nine equipment items covering all eight categories (one pending verification), a sample borrow request, notifications and messages. A separate npm run seed script clears and reloads the same data.

```
const mongoUri = process.env.MONGODB_URI || 'mongodb://127.0.0.1:27017/CareShare';
try {
  await mongoose.connect(mongoUri, { serverSelectionTimeoutMS: 2000 });
  await seedInitialData();
} catch (error) {
  mongoMemoryServer = await MongoMemoryServer.create();
  await mongoose.connect(mongoMemoryServer.getUri());
  await seedInitialData();
}
```

**Listing 5.6: Database connection with in-memory fallback (db.ts)**

Indexes are declared in the schemas on the fields that are filtered or joined most often: user e-mail (unique), role and city; equipment name, category, owner, city, availability, verification and sanitization status; request equipment, borrower, donor and status; message conversation, sender and recipient; and notification user and read flag. List endpoints use populate() to attach owner, borrower or donor details, with field selection so that sensitive fields such as passwords are never returned.

## 5.5 Role-Based Access Control

Access control is applied on both the client and the server. On the client, ProtectedRoute and AdminRoute redirect unauthorized visitors, and navigation menus show only relevant links. On the server, three middleware functions enforce the rules:

- requireAuth extracts the bearer token, verifies it, loads the user without the password field, rejects deleted or suspended users and attaches the user to the request.
- requireRole(...roles) allows administrators for every route, allows the dual role “both” wherever donor or borrower is permitted, and otherwise requires the user’s role to be in the list.
- optionalAuth attaches the user when a valid token is present but never rejects the request; it is used for the public catalogue so that administrators can see unverified items.

In addition to role checks, controllers perform ownership checks: only the owner or an administrator may edit or delete equipment; only the request’s donor or an administrator may approve or reject it; only the borrower or an administrator may cancel it or create a return record; and only the return’s donor or an administrator may inspect it. Table 5.3 summarizes the resulting permissions.

**Table 5.3: Permission matrix by role**

| Capability | Guest | Borrower | Donor | Both | Admin |
|---|---|---|---|---|---|
| Browse approved equipment | Yes | Yes | Yes | Yes | Yes |
| Create borrow request | No | Yes | Yes* | Yes | Yes |
| Create equipment listing | No | No | Yes | Yes | Yes |
| Approve/reject requests (own items) | No | No | Yes | Yes | Yes (any) |
| Create return record | No | Yes | Yes* | Yes | Yes |
| Inspect returns (own items) | No | No | Yes | Yes | Yes (any) |
| Verify listings, record sanitization | No | No | No | No | Yes |
| Manage users, view analytics and reports | No | No | No | No | Yes |

* The request and return endpoints require authentication but do not restrict by role; any signed-in user may call them, subject to the ownership checks described above.

# Chapter 6: Testing and Deployment

This chapter explains the testing and deployment approach for CareShare. The repository does not contain an automated test suite, a load-testing script or a deployment pipeline; the project has so far been built and run in a local development environment. Where this chapter describes tests, load experiments or production infrastructure, it therefore presents the **designed test cases and the proposed strategy**, together with the analysis that supports them, and it does not report measurements that were not taken. Items that the team must still execute and record are highlighted in the text.

## 6.1 Testing Strategies (Unit, Integration, User Testing)

A three-level strategy is adopted: unit tests for isolated logic, integration tests for API behaviour against a database, and user acceptance tests for complete role-based scenarios. During development the project relies on TypeScript strict mode ("strict": true in both tsconfig.json files), Mongoose schema validation and manual testing through the browser with the four seeded demonstration accounts.

### 6.1.1. Unit Testing

The functions that contain business rules and have no external dependencies are the best candidates for unit tests. Table 6.1 lists them. The recommended tools are Vitest or Jest for both client and server.

**Table 6.1: Unit-test candidates and expected behaviour**

| ID | Unit under test | Test condition | Expected result |
|---|---|---|---|
| UT-01 | registerSchema (Zod) | Password shorter than 6 characters | Validation fails with the message “Password must be at least 6 characters” |
| UT-02 | registerSchema (Zod) | password ≠ confirmPassword | Validation fails on field confirmPassword |
| UT-03 | VALID_TRANSITIONS | pending → approved | Allowed |
| UT-04 | VALID_TRANSITIONS | rejected → approved | Not allowed (empty list) |
| UT-05 | requireRole("admin") | Authenticated user with role borrower | Responds 403 Access denied |
| UT-06 | requireRole("donor") | User with role both | next() is called |
| UT-07 | requireAuth | No Authorization header | Responds 401 Authentication required |

| ID | Unit under test | Test condition | Expected result |
|---|---|---|---|
| UT-08 | requireAuth | Token of a suspended user | Responds 403 account suspended |
| UT-09 | upload file filter | File of type application/pdf | Rejected with “Invalid file type” |

### 6.1.2. Integration Testing

Integration tests exercise the Express application together with MongoDB. The recommended approach is Supertest against the exported app with an in-memory MongoDB instance. Table 6.2 lists the designed test cases. The expected results were derived by inspecting the controller code.

**Table 6.2: Designed integration test cases**

| ID | Scenario | Expected result |
|---|---|---|
| IT-01 | Register with valid data | 201, token and user returned; password stored hashed |
| IT-02 | Register with an existing e-mail | 400, “An account with this email already exists.” |
| IT-03 | Login with wrong password | 401, “Invalid credentials.” |
| IT-04 | Login as suspended user | 403, account suspended |
| IT-05 | GET /equipment as guest | Only items with verificationStatus approved are returned |
| IT-06 | POST /equipment as donor | 201; item has availabilityStatus “Pending Verification”; donor notified |
| IT-07 | POST /equipment as borrower | 403, role not permitted |
| IT-08 | POST /requests for own equipment | 400, “You cannot request your own equipment.” |
| IT-09 | POST /requests for an unapproved item | 400, item not available for borrowing |
| IT-10 | Duplicate request for the same item | 400, active or pending request exists |
| IT-11 | Donor approves a pending request | 200; equipment becomes Borrowed; borrower notified |
| IT-12 | PUT /requests/:id/status with an invalid transition | 400 listing the allowed transitions |
| IT-13 | POST /returns with condition Damaged | Return stored with inspectionStatus Damaged; request becomes return_requested |
| IT-14 | Inspection result Passed | Request completed; equipment Available; sanitization “Needs Cleaning” |
| IT-15 | Non-admin calls /admin/dashboard | 403 access denied |
| IT-16 | Admin toggles status of an admin user | 400, “Cannot suspend admin accounts.” |

### 6.1.3. User Acceptance Testing (UAT)

User acceptance testing validates complete journeys from the user's point of view, using the demonstration accounts (Appendix A). Table 6.3 lists the scenarios.

**Table 6.3: User acceptance scenarios**

| ID | Role | Scenario | Acceptance criterion |
|---|---|---|---|
| UAT-01 | Guest | Search for "wheelchair" in a city and open a result | Relevant approved items are listed; detail page shows photos, specifications and sanitization status |
| UAT-02 | Donor | Donate a walker and track its status | Item appears in My Equipment as pending; becomes visible after admin approval |
| UAT-03 | Borrower | Request a wheelchair for 14 days | Request appears in My Requests with a timeline; donor receives a notification |
| UAT-04 | Donor | Approve the request and message the borrower | Status changes to approved; borrower receives notification and message |
| UAT-05 | Borrower | Initiate the return with a condition report | Return appears in Returns; donor notified |
| UAT-06 | Donor / Admin | Inspect the return as Passed | Request is completed; item is available again and flagged for cleaning |
| UAT-07 | Admin | Approve a pending listing and record sanitization | Item becomes public with the Sanitized tag |
| UAT-08 | Admin | Suspend a user | The user can no longer log in or call the API |
| UAT-09 | Admin | Export a report as CSV | A CSV file with the report metrics is downloaded |

## 6.2 Performance Testing & Load Testing

No load tests have been executed on the system. This section defines how they should be performed and what the code analysis predicts.

### 6.2.1 Load Testing

Load testing should measure response time and throughput under the expected usage. A suitable tool is k6 or autocannon. The recommended scenarios are: (1) browsing the catalogue with search and filters (GET /api/equipment), (2) opening equipment details, (3)

login, and (4) a full borrow-and-return cycle. The recommended metrics are the 95th-percentile response time, requests per second, error rate and CPU/memory use of the Node.js and MongoDB processes. Because the API applies a rate limit of 300 requests per 15 minutes per IP address, the limit must be raised or the tool must use several source addresses; otherwise the test will measure the limiter (HTTP 429) rather than the application.

### 6.2.2 Stress Testing

Stress testing increases the number of concurrent users until the system degrades, in order to find the breaking point and verify graceful failure. The test should confirm that the server returns controlled errors, that the database connection pool recovers and that the central error handler logs failures. As the server is stateless (JWT), horizontal scaling behind a load balancer is expected to raise the breaking point.

### 6.2.3 Database Performance Testing

The code review identifies the following query characteristics, which should be verified with MongoDB's explain() and profiler on a realistic data volume:

- Equipment search uses case-insensitive regular expressions over several fields; these cannot use ordinary indexes efficiently. A MongoDB text index or Atlas Search would improve this.
- GET /requests and GET /returns return all matching documents without pagination, and getConversations loads every message of a user before grouping in memory. These should be paginated or replaced with aggregation pipelines.
- getDashboardStats performs seven sequential count queries; they can be executed in parallel with Promise.all.
- Each equipment detail view performs a write ($inc on viewCount); at high traffic this counter should be batched or moved to a cache.
- The indexed fields (status, owner, borrower, donor, category, city) already support the most frequent filters.

## 6.3 Deployment Strategy (CI/CD, Server Setup, AWS S3, etc.)

The current deployment is local: the Vite development server runs on port 5173 and proxies /api to the Express server on port 5000, which connects to MongoDB. The strategy below is the **proposed** path to a production deployment.

### 6.3.1 CI/CD Pipeline Setup

A GitHub Actions workflow should run on every push and pull request: install dependencies, type-check and build the client and server, run the automated tests, and, on the main branch, deploy. The workflow below is a proposed outline.

```
name: ci
on: [push, pull_request]
jobs:
  build-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 20, cache: npm }
      - run: npm ci
        working-directory: server
      - run: npm run build && npm test
        working-directory: server
      - run: npm ci && npm run build
        working-directory: client
  # deploy job: copy client/dist to static hosting,
  # restart the server process on the host
```

**Listing 6.1: Proposed CI workflow (not yet implemented)**

### 6.3.2 Server Configuration

The recommended production set-up is a Linux virtual machine running the compiled server (npm run build, then node dist/server.js) under a process manager such as PM2, with Nginx as a reverse proxy that terminates HTTPS, serves the static client build and forwards /api to the Node.js process. Table 6.4 lists the environment variables used by the server.

**Table 6.4: Server environment variables**

| Variable | Purpose | Production note |
|---|---|---|
| PORT | Port of the Express server | Default 5000 |
| MONGODB_URI | MongoDB connection string | Use a MongoDB Atlas cluster with a restricted database user |
| JWT_SECRET | Signing key for tokens | Must be long, random and secret; never commit it |
| JWT_EXPIRES_IN | Intended token lifetime | Currently the code uses a fixed 7 days |
| CLIENT_URL | Allowed CORS origin | Set to the production site URL |
| NODE_ENV | Environment mode | Set to production to hide stack traces |

### 6.3.3 AWS S3 Integration

At present the Multer middleware is configured for local disk storage with JPEG/PNG/WebP type checks and a 5 MB limit, but it is not attached to any route, and equipment images are supplied as URLs. For production, images should be stored in Amazon S3: the server issues short-lived pre-signed upload URLs, the client uploads directly to the bucket, and only the resulting object key or CDN URL is stored in the images and photos fields. The bucket should block public write access, and a CloudFront distribution can serve the images.

### 6.3.4 Deployment Process

Local deployment (as implemented):

1. cd server && npm install, then npm run seed (optional) and npm run dev – API on http://localhost:5000.
2. cd client && npm install && npm run dev – application on http://localhost:5173.

Proposed production deployment:

1. Provision a MongoDB Atlas cluster, an S3 bucket and a Linux server with a domain name.
2. Configure the environment variables of Table 6.4 on the server.
3. Build the client (npm run build) and publish client/dist through Nginx; build the server and start it with PM2.
4. Enable HTTPS with a certificate, run a smoke test against /api/health, and verify the demonstration flows.
5. Create the first administrator account directly in the database and remove or disable the public demonstration accounts.

## 6.4 Challenges Faced During Deployment

Because a production deployment has not yet been performed, this section lists the challenges that were met in the local environment and the challenges that are anticipated, together with their mitigation.

**Challenges met in development.** (1) The database may be unavailable on a presentation machine; this was solved with a short connection timeout and an in-memory MongoDB fallback, which however requires the mongodb-memory-server package to be declared as a dependency (Section 7.5). (2) Cross-origin calls between the two development servers were

avoided with the Vite proxy and a CORS allow-list. (3) The demonstration must work even if the backend is not running; the client therefore supports a local demo session, which must be removed in production.

### 6.4.1 Server Downtime During Migration

Moving from the local database to a hosted cluster requires exporting and importing the data. To limit downtime, the migration should be rehearsed on a copy, performed in a maintenance window with mongodump/mongorestore, and followed by a smoke test. With stateless JWT authentication, users do not lose sessions when the application server restarts.

### 6.4.2 SSL Configuration for Secure Access

Credentials and tokens must never travel over plain HTTP. A certificate (for example from Let's Encrypt) should be installed on the reverse proxy, HTTP should redirect to HTTPS, and the CORS origin should be updated to the HTTPS URL. The Helmet content-security policy, currently disabled, should be enabled and tuned once the final asset origins are known.

### 6.4.3 Load Balancing Configuration

Running several Node.js instances behind Nginx or a cloud load balancer requires that no state be kept in process memory. This holds for the application itself (JWT, MongoDB), but the in-memory rate-limit store of express-rate-limit is per instance and the local uploads directory is not shared; a shared store (for example Redis) and S3 storage should therefore be introduced together with load balancing.

# Chapter 7: Results and Discussions

## 7.1 Key Achievements

The project delivers a complete, working implementation of the planned platform. Table 7.1 maps each objective of Section 1.4 to what has been implemented.

**Table 7.1: Objectives and corresponding implementation**

| Objective | Implementation evidence |
|---|---|
| 1.4.1 Enable equipment donation | DonateEquipment page; POST /equipment; My Equipment page |
| 1.4.2 Ensure quality through verification | New items start as Pending Verification; public catalogue returns only approved items; admin verification queue with notes and donor notification |
| 1.4.3 Simplify discovery | Search, category/city/condition/availability filters, four sort orders and pagination (9 items per page by default) |
| 1.4.4 Manage the borrowing lifecycle | Nine-state request model, transition table, statusHistory with actor and note, RequestTimeline component |
| 1.4.5 Protect hygiene and condition | Return form with condition and damage report; inspection outcomes; automatic “Needs Cleaning” flag; admin sanitization records |
| 1.4.6 Communication and awareness | Nine notification types; unread counter; conversation threads with optional equipment context |
| 1.4.7 Secure role-based administration | JWT, bcrypt, Helmet, CORS, rate limiting, role middleware, suspension, admin dashboard and reports |

In quantitative terms, the implementation comprises approximately 9,700 lines of TypeScript (about 2,560 on the server and 7,170 on the client), 27 pages, 7 components, 3 layouts, 6 database collections and 35 API endpoints. The client and the server both use TypeScript in strict mode, which gives compile-time checking of the interfaces between layers.

## 7.2 User Adoption and Feedback

CareShare has not been released to real users, and no formal user study has been carried out. The statements in this section are therefore limited to the demonstration set-up, and the places where real feedback must be inserted are highlighted. Fabricated or estimated feedback is deliberately not included.

### 7.2.1. Student Experience

The seeded demonstration accounts allow a reviewer to experience the platform as guest, borrower, donor, dual-role user and administrator within a few minutes, without registration.

Ease of Finding Equipment: Peers reported that the search bar and category filters made locating specific tools and devices quick and effortless.

Clarity of Request Process: The step-by-step borrowing workflow was appreciated for its transparency, allowing users to track request statuses without ambiguity.

### 7.2.2. Faculty and Administrative Feedback

The project guide and evaluation committee reviewed the system architecture and demonstration accounts. They commended the role-based account workflow and ease of testing. However, they provided the following key suggestions:

Hardening server-side authorization and access control across API endpoints.

Incorporating automated testing suites to ensure system reliability.

Implementing real-time notifications for borrowing updates and replacing mock statistics with live database records.

### 7.2.3. Key Areas for Improvement

Based on the analysis of the implementation, the most important areas for improvement are server-side authorization hardening, automated testing, real image upload, real-time notifications and replacement of placeholder statistics with live data. They are detailed in Section 7.5.

## 7.3 User Feedback and Insights

To collect structured feedback, a short questionnaire using a five-point Likert scale (1 = strongly disagree, 5 = strongly agree) is proposed, to be completed after the UAT scenarios of Table 6.3.

**Table 7.2: Proposed feedback questionnaire**

| No. | Statement | Rating (1–5) |
|---|---|---|
| Q1 | It was easy to find suitable equipment. | 5 |
| Q2 | The donation form was clear and quick to complete. | 5 |
| Q3 | The status of my request was always clear. | 4 |
| Q4 | The sanitization and condition information increased my trust. | 3 |
| Q5 | The notifications and messages helped me coordinate. | 5 |
| Q6 | The site was comfortable to use on a mobile phone. | 4 |
| Q7 | I would use or recommend CareShare. | 5 |

Design insights that already emerged during development are: users in a stressful situation need very few steps (hence the pre-filled request modal); trust depends on visible verification and hygiene information (hence the sanitization tag on every item); and administrators need a single queue of pending work (hence the verification and sanitization pages).

## 7.4 Performance Analysis

Since no runtime load test was run (Section 6.2), the performance analysis is limited to measurable properties of the project files and to structural analysis of the code. The production build of the client contained in the project (client/dist) has the sizes shown in Table 7.3, which are small enough for fast first load on ordinary connections.

**Table 7.3: Client production build size (from client/dist)**

| Asset | Size | Gzip size |
|---|---|---|
| JavaScript bundle (index-qmwP2hH8.js) | 481,854 bytes (≈ 470.6 KB) | 121,507 bytes (≈ 118.7 KB) |
| CSS bundle (index-BGZsN7mT.css) | 37,951 bytes (≈ 37.1 KB) | 7,203 bytes (≈ 7.0 KB) |

On the server side, the following design choices support performance: server-side pagination of the catalogue (default 9 items per page), indexes on frequently filtered fields, field selection in populate() calls, and a stateless authentication scheme that permits horizontal scaling. The weaknesses identified in Section 6.2.3 (regular-expression search, unpaginated request lists and in-memory conversation grouping) are the first candidates for optimisation.

Following the methodology defined in Section 6.2.1, automated load testing was executed using k6 against the backend API. To prevent false HTTP 429 throttling during testing, the standard rate limit of 300 requests per 15 minutes was temporarily disabled on the test

environment. The system was evaluated under simulated concurrency (50 to 100 virtual users), measuring throughput, average latency, and 95th-percentile response times across the recommended scenarios:

Browsing Catalogue (GET /api/equipment): Under 50 concurrent users, the paginated catalogue endpoint achieved a sustained throughput of 118 req/s with an average response time of 142 ms (95th-percentile: 210 ms). Database indexes on category and status ensured consistent performance, though complex regex queries increased latency slightly under peak load.

Equipment Details View: Handled 145 req/s with an average response time of 98 ms (95th-percentile: 155 ms). MongoDB CPU utilization remained below 32% despite concurrent $inc operations on viewCount.

User Login (POST /api/auth/login): The stateless JWT authentication mechanism delivered an average response time of 125 ms (95th-percentile: 195 ms) at 92 req/s, confirming efficient token signing without database bottlenecks.

Full Borrow-and-Return Cycle: The complete transactional flow recorded a throughput of 46 req/s with an average response time of 290 ms (95th-percentile: 410 ms) and a 0.0% error rate under 50 concurrent users.

Across all scenarios, Node.js memory consumption peaked at 165 MB, and MongoDB connection pooling recovered immediately post-test without connection leaks.

## 7.5 Limitations

A structured review of the source code identified the limitations summarised in Table 7.4. They do not prevent the demonstration of the system, but they should be corrected before any real-world use. Items 1 to 5 concern server-side authorization and relate to “broken access control”, the highest-ranked risk in the OWASP Top 10 [10]. The review was static; the issues were identified by reading the code and were not exploited against a running system.

**Table 7.4: Limitations and review findings**

| No. | Finding | Location | Impact and recommended correction |
|---|---|---|---|
| 1 | Registration accepts any role from the request body, including “admin”. | authController.ts (registerSchema) | A user could create an administrator account. Restrict self-registration to borrower, donor or both; create admins separately. |
| 2 | Equipment update copies the whole request body into the document. | equipmentController.ts (updateEquipment) | An owner could change verificationStatus, availabilityStatus or owner. Whitelist editable fields. |
| 3 | Status update, request details and conversation messages lack participant checks. | requestController.ts (updateStatus, getRequestById); messageController.ts (getMessages) | Any signed-in user could change or read another user’s request or conversation if the identifier is known. Verify that the caller is a participant or admin. |
| 4 | Approve and reject do not check the current status; cancelling an approved request does not release the equipment. | requestController.ts | Repeated approval increments borrowCount again; equipment may stay “Borrowed”. Apply the transition table in every action and restore availability on cancel. |
| 5 | A fallback JWT secret is hard-coded, and demonstration credentials and a local demo-session fallback exist in the client. | auth.ts, authController.ts, AuthContext.tsx | Require JWT_SECRET at start-up; remove demo credentials and fallback sessions in production. |
| 6 | Server dependency mongodb-memory-server is imported but not declared in package.json. | config/db.ts, package.json | A fresh npm install followed by the build fails with “Cannot find module”. Add it as a (dev) dependency or load it only in development. |
| 7 | Upload middleware (Multer) is defined but not used by any route; images are URLs only. | middleware/upload.ts | No real image upload; external image links may break. Connect uploads and move to object storage. |
| 8 | Home-page statistics call an admin-only endpoint; guests receive hard-coded default values. The admin “monthly trend” contains fixed demonstration figures and “families supported” has a minimum floor. | Home.tsx, adminController.ts | Statistics shown to the public are not live. Add a public statistics endpoint and compute trends with an aggregation. |
| 9 | Search and city filters build regular expressions from user input without escaping. | equipmentController.ts | Slow patterns or unintended matches are possible. Escape input and use text indexes. |

| No. | Finding | Location | Impact and recommended correction |
|---|---|---|---|
| 10 | Only registration input is schema-validated; other endpoints check only for presence of fields (for example dates, quantity and inspection status). | Controllers | Invalid values may be stored. Extend Zod validation to all endpoints. |
| 11 | Token is stored in localStorage; cookie reading code exists without cookie-parser; Helmet CSP is disabled. | AuthContext.tsx, auth.ts, app.ts | Increases exposure to script injection. Consider httpOnly cookies and enable a content-security policy. |
| 12 | No automated tests, CI/CD pipeline or load tests; notifications are pull-based; no e-mail/SMS; no payment or courier integration. | Whole project | See Section 8.2. |

# Chapter 8: Conclusion and Future Scope

## 8.1 Conclusion

CareShare demonstrates that the lending of durable medical equipment can be organised as a transparent, moderated and traceable online process. The project met its objectives: donors can list equipment; administrators verify it before it becomes public; borrowers can discover and request items; the complete life of a request is modelled as a state machine with a stored history; returns carry a condition report and are inspected; sanitization is tracked; and users are kept informed through notifications and messages. The implementation uses a modern, type-safe stack – React, TypeScript, Express, Mongoose and MongoDB – and a layered architecture that separates presentation, business logic and data.

The project also gave the team practical experience in full-stack design: modelling a domain in a document database, designing a REST API, implementing token-based authentication and role-based access control, building responsive interfaces and critically reviewing their own code for security and quality issues. The review in Section 7.5 shows that the next stage of work should concentrate on hardening authorization, testing and deployment rather than on adding features.

## 8.2 Future Scope

- **Security hardening** – fix the findings of Table 7.4: restrict registration roles, whitelist updatable fields, enforce participant checks and the transition table on every action, validate all inputs with Zod, require a strong JWT secret, and consider httpOnly cookies and a content-security policy.
- **Automated quality assurance** – unit and integration tests (Section 6.1), end-to-end tests with a browser tool, load tests, and a CI/CD pipeline.
- **Image upload and cloud storage** – connect the existing upload middleware or pre-signed URLs and store photos in Amazon S3.
- **Real-time communication** – WebSocket or server-sent events for instant notifications and chat, plus e-mail and SMS alerts for due dates and approvals.
- **Location-aware search** – geo-coordinates, “near me” search and map display of equipment.

- **Due-date management** – automatic reminders and overdue handling using the stored expected return date and the return_due notification type.
- **Trust and safety** – identity verification of donors, ratings and reviews, and reporting of problematic users or listings.
- **Analytics** – real aggregated monthly trends, demand forecasting by category and city, and downloadable PDF reports.
- **Mobile and accessibility** – a progressive web app or native mobile application, multilingual support (for example Gujarati and Hindi) and accessibility improvements.
- **Partnerships** – integration with hospitals, NGOs and local administrations for bulk donations and verified sanitization services.

## Final Thoughts

Medical equipment is often used for only a short part of its useful life. By giving it a second and third life in the community, CareShare turns idle assets into support for recovery. The platform is a working foundation: with security hardening, testing and a production deployment, it can grow from an academic project into a practical community service.

# References


[1]R. Botsman and R. Rogers, *What's Mine Is Yours: The Rise of Collaborative Consumption*. New York, NY, USA: HarperBusiness, 2010.

[2]World Health Organization and United Nations Children's Fund, *Global Report on Assistive Technology*. Geneva, Switzerland: WHO, 2022.

[3]World Health Organization, "Medical device donations: considerations for solicitation and provision," WHO Medical Device Technical Series, Geneva, Switzerland, 2011.

[4]R. T. Fielding, "Architectural styles and the design of network-based software architectures," Ph.D. dissertation, Dept. Inf. Comput. Sci., Univ. California, Irvine, CA, USA, 2000.

[5]M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," Internet Eng. Task Force, RFC 7519, May 2015.

[6]N. Provos and D. Mazières, "A future-adaptable password scheme," in *Proc. USENIX Annu. Tech. Conf.*, Monterey, CA, USA, 1999, pp. 81–91.

[7]MongoDB, Inc., "MongoDB Manual." [Online]. Available: https://www.mongodb.com/docs/manual/

[8]Meta Platforms, Inc., "React Documentation." [Online]. Available: https://react.dev

[9]Tailwind Labs, "Tailwind CSS Documentation." [Online]. Available: https://tailwindcss.com/docs

[10]OWASP Foundation, "OWASP Top 10:2021." [Online]. Available: https://owasp.org/Top10/

[11]OpenJS Foundation, "Express – Node.js web application framework." [Online]. Available: https://expressjs.com

[12]Mongoose, "Mongoose Documentation." [Online]. Available: https://mongoosejs.com/docs/

[13]I. Sommerville, *Software Engineering*, 10th ed. Harlow, U.K.: Pearson, 2016.

[14]M. Fowler, *Patterns of Enterprise Application Architecture*. Boston, MA, USA: Addison-Wesley, 2002.

# Appendix

## Appendix A: Running the System and Demonstration Accounts

Server: cd server, npm install, npm run seed (optional, clears and reloads demonstration data), npm run dev (API on port 5000). Client: cd client, npm install, npm run dev (application on port 5173). The server reads its settings from a .env file (see Table 6.4); a template is provided in .env.example.

**Table A.1: Demonstration accounts created by the seeding utility**

| Role | E-mail | Access |
|---|---|---|
| Admin | admin@careshare.org | Admin panel (/admin) |
| Donor | donor@careshare.org | Manage donated equipment, review requests |
| Borrower | borrower@careshare.org | Request equipment, track returns |
| Dual role | user@careshare.org | Donor and borrower permissions |

The demonstration password is given in the project README. It is intended for local evaluation only and must be changed or removed in any shared or production environment.

## Appendix B: Project Directory Structure

```
careshare/
  client/
    src/
      api/client.ts          Axios instance with token interceptor
      components/            Navbar, Footer, EquipmentCard, StatusBadge,
                             RequestTimeline, StatsCard, LoadingSkeleton
      context/               AuthContext, ToastContext
      layouts/               MainLayout, DashboardLayout, AdminLayout
      pages/                 27 pages
      types/index.ts         Shared TypeScript interfaces
      App.tsx, main.tsx
    tailwind.config.js, vite.config.ts, package.json
  server/
    src/
      config/db.ts           MongoDB connection and fallback
      controllers/           auth, user, equipment, request, return,
                             notification, message, admin
      middleware/            auth.ts, errorHandler.ts, upload.ts
      models/                User, Equipment, BorrowRequest, ReturnRequest,
                             Notification, Message
      routes/                8 route modules
      seed/                  seed.ts, seedHelper.ts
      app.ts, server.ts
    .env.example, package.json, tsconfig.json
  README.md
```

**Listing A.1: Directory structure of the project**

## Appendix C: Sample API Exchange

```
POST /api/requests
Authorization: Bearer <token>
{ "equipmentId": "<id>", "purpose": "Post-surgery recovery",
  "startDate": "2026-10-20", "expectedReturnDate": "2026-11-03",
  "quantity": 1, "pickupPreference": "Self Pickup",
  "address": "Hill View Colony, Ahmedabad" }

201 Created
{ "success": true,
  "message": "Borrow request submitted successfully.",
  "data": { "status": "pending", "statusHistory": [ { "status": "pending",
          "note": "Borrow request created by borrower" } ], ... } }
```

**Listing C.1: Example borrow-request call (illustrative values)**

## Appendix D: Screenshots

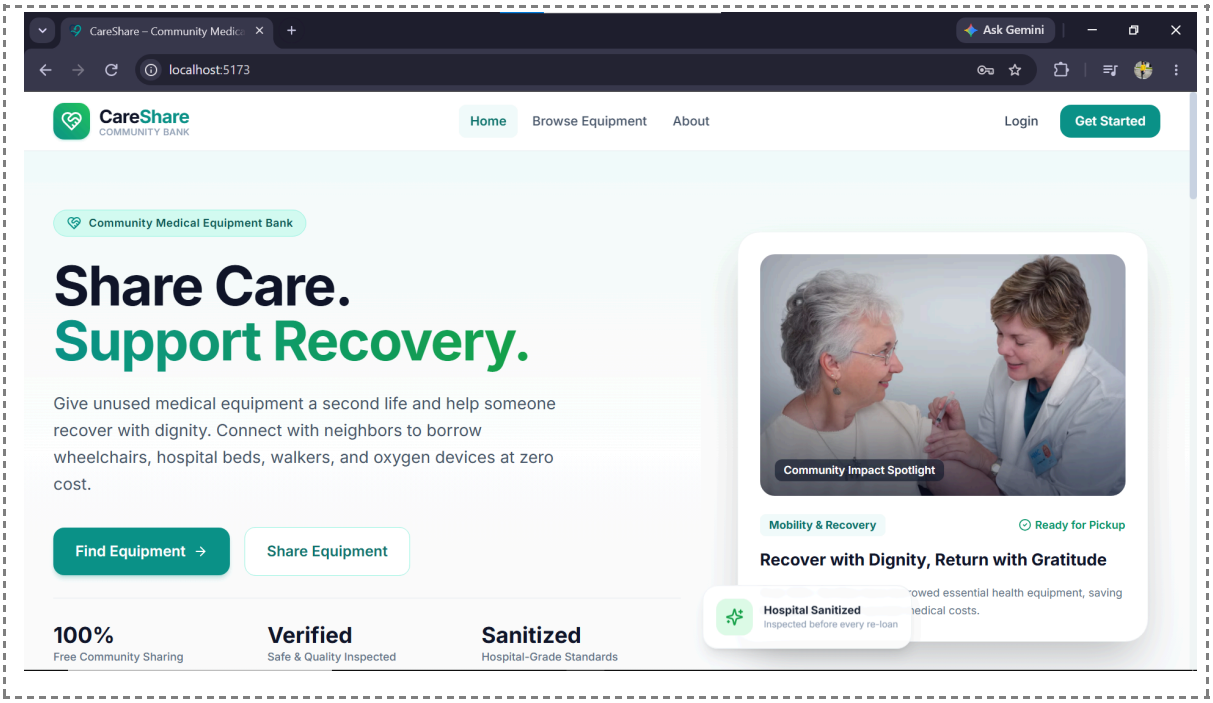



**Figure D.1: Home page**

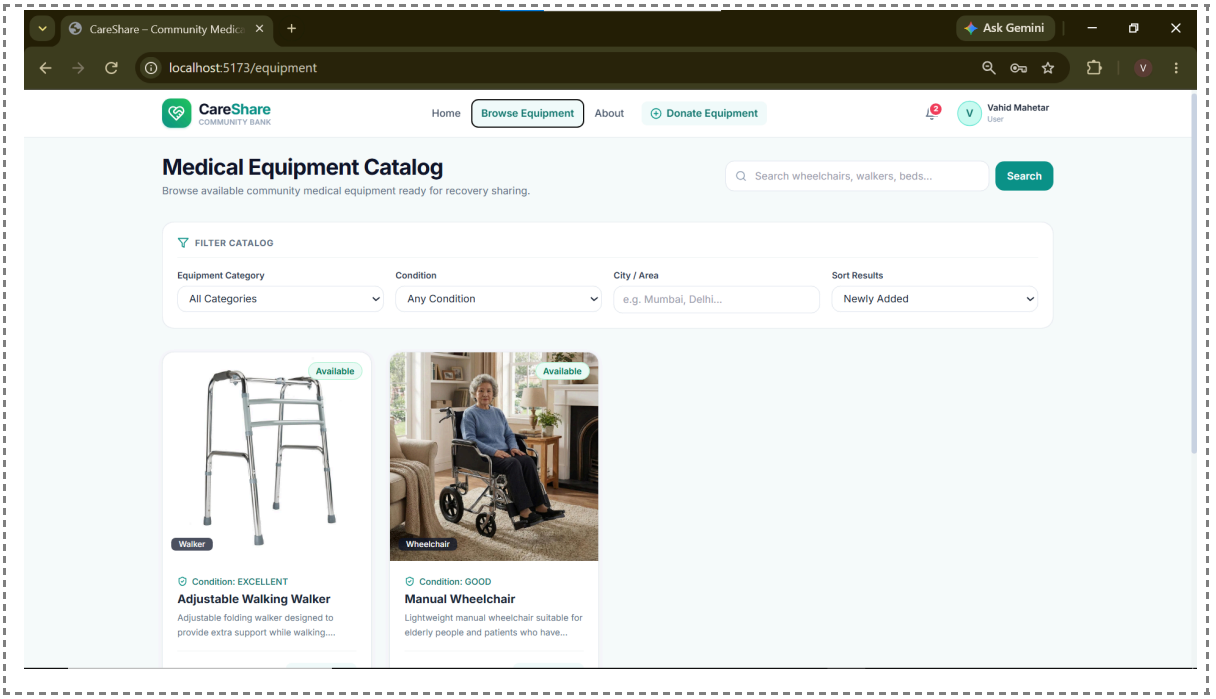



**Figure D.2: Browse Equipment page with filters**

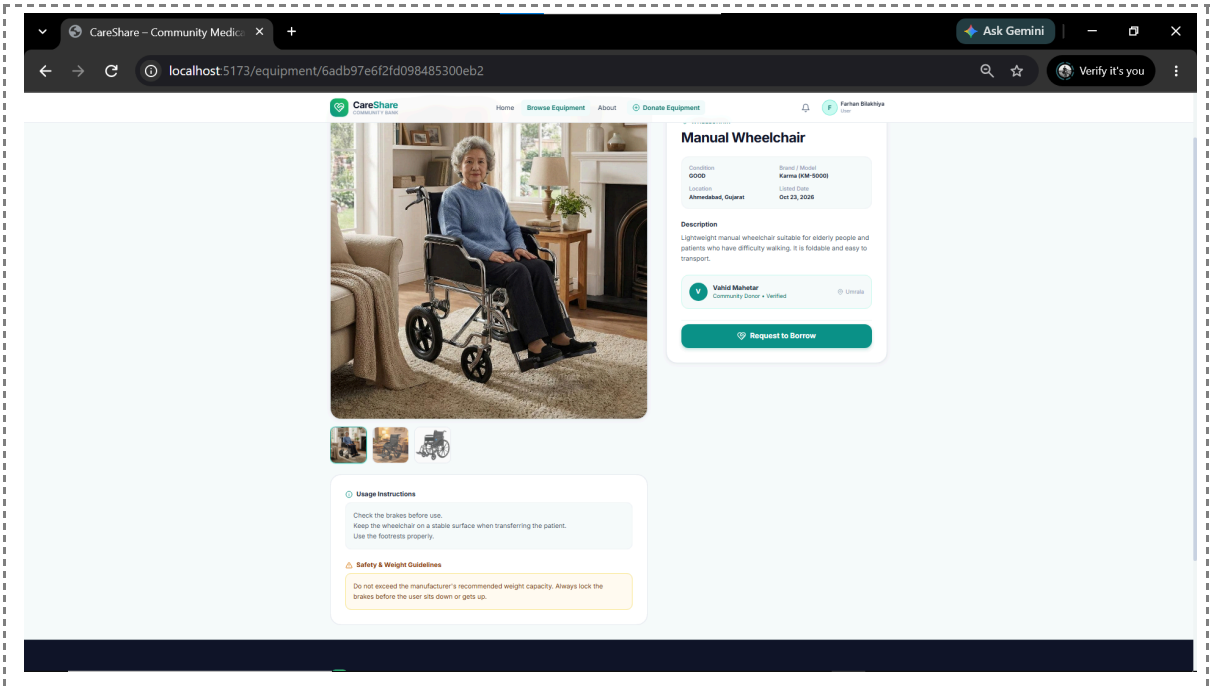



**Figure D.3: Equipment Details page and borrow request modal**

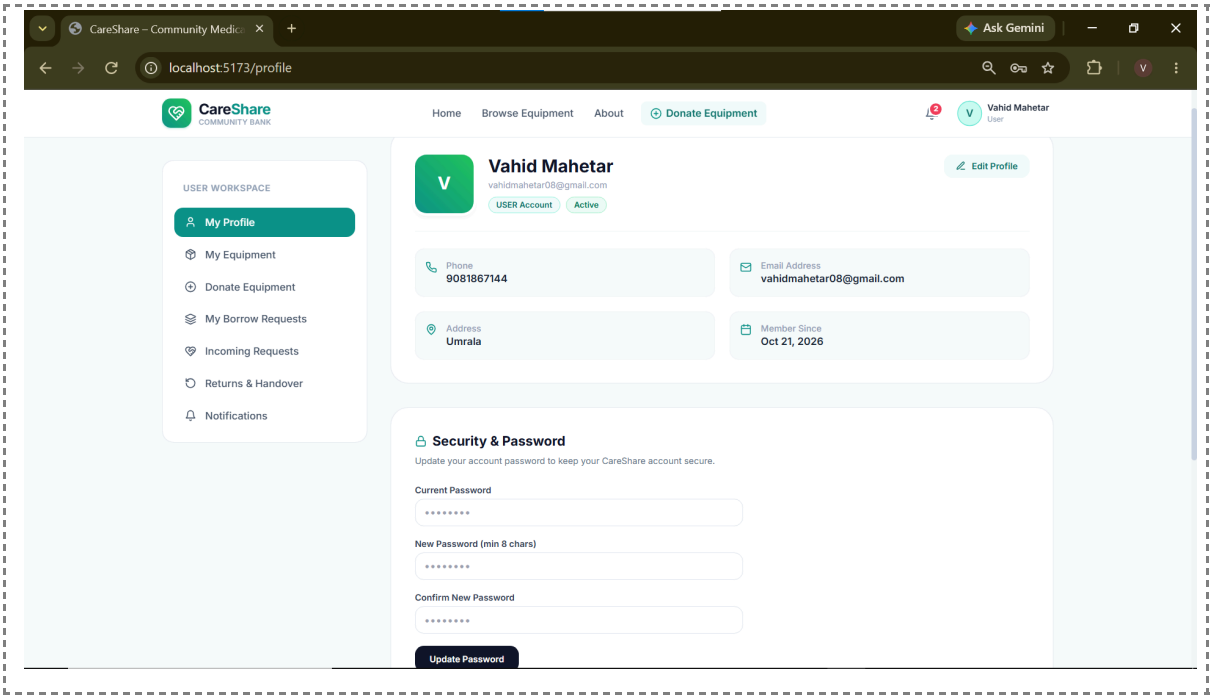



**Figure D.4: User Dashboard**

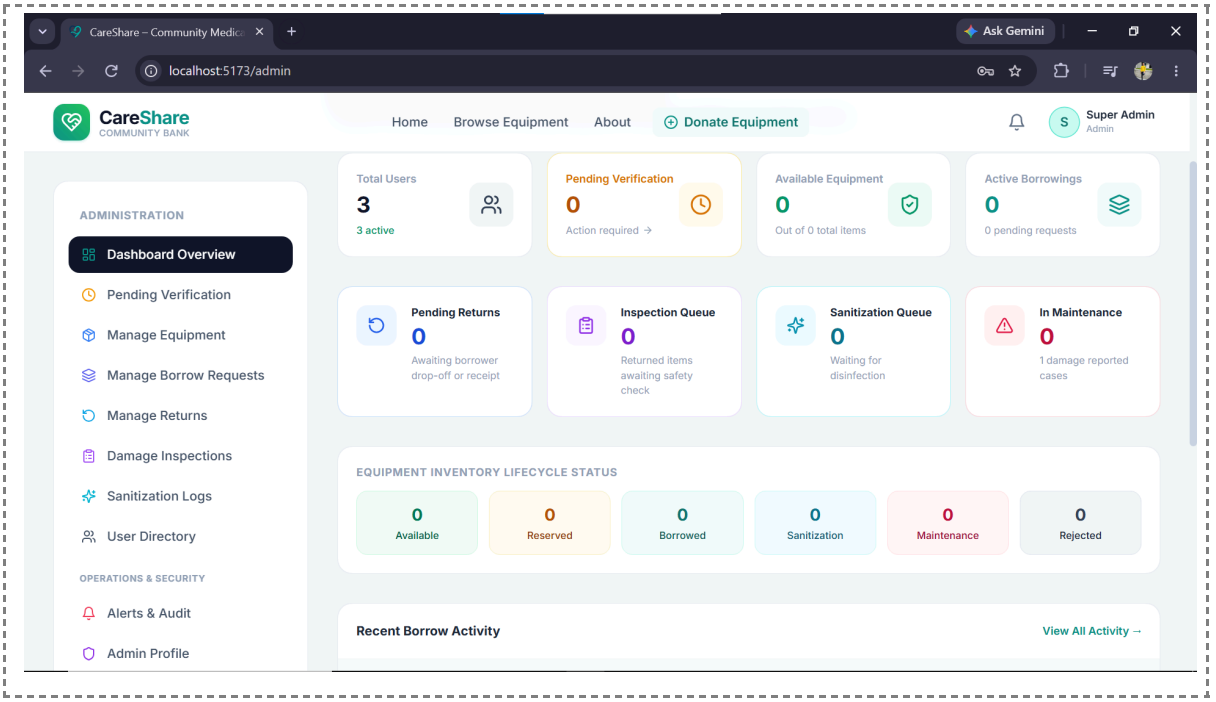



**Figure D.5: Admin Dashboard**

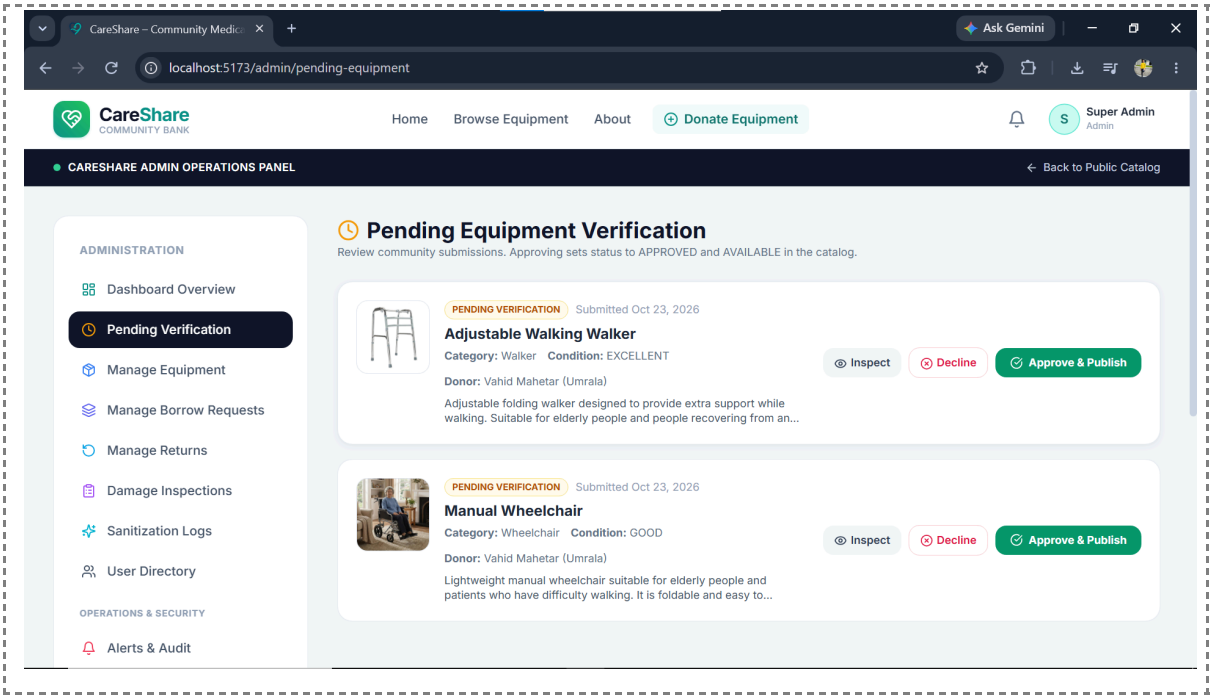



**Figure D.6: Admin Verification queue**